Big Data Processing

Lecture Week 7 – problem solving

Hive and Pig

RMIT UNIVERSITY

Q1 - In 2008, the database community criticise "MapReduce is a step backwards". Among their arguments, which of the followings is not correct?

- MapReduce does not support transactions.
- The idea of partitioning data and processing them in parallel in MapReduce is not new.
- MapReduce does not support schemas.
- MapReduce does not support bulk load but it supports data update.
- MapReduce scan the data set rather than using index to improve the data access.

Is MapReduce a Major Step Backwards?

MapReduce may be a good idea for certain types of general-purpose computations

Is MapReduce a Major Step Backwards?

MapReduce may be a good idea for certain types of general-purpose computations,

But to the database community in 2008…

Is MapReduce a Major Step Backwards?

MapReduce may be a good idea for certain types of general-purpose computations,

But to the database community in 2008 …

1. MapReduce is a step backwards in database access
2. A sub-optimal implementation, in that it uses brute force instead of indexing
3. Not novel at all -- it represents a specific implementation of well known techniques developed nearly 25 years ago
4. Missing most of the features that are routinely included in current DBMS
5. Incompatible with all of the tools DBMS users have come to depend on

Is MapReduce a Major Step Backwards?

MapReduce may be a good idea for certain types of general-purpose computations,

But to the database community in 2008 …

1. MapReduce is a step backwards in database access
2. A sub-optimal implementation, in that it uses brute force instead of indexing
3. Not novel at all -- it represents a specific implementation of well known techniques developed nearly 25 years ago
4. Missing most of the features that are routinely included in current DBMS
5. Incompatible with all of the tools DBMS users have come to depend on

1. MapReduce is a step backwards in database access

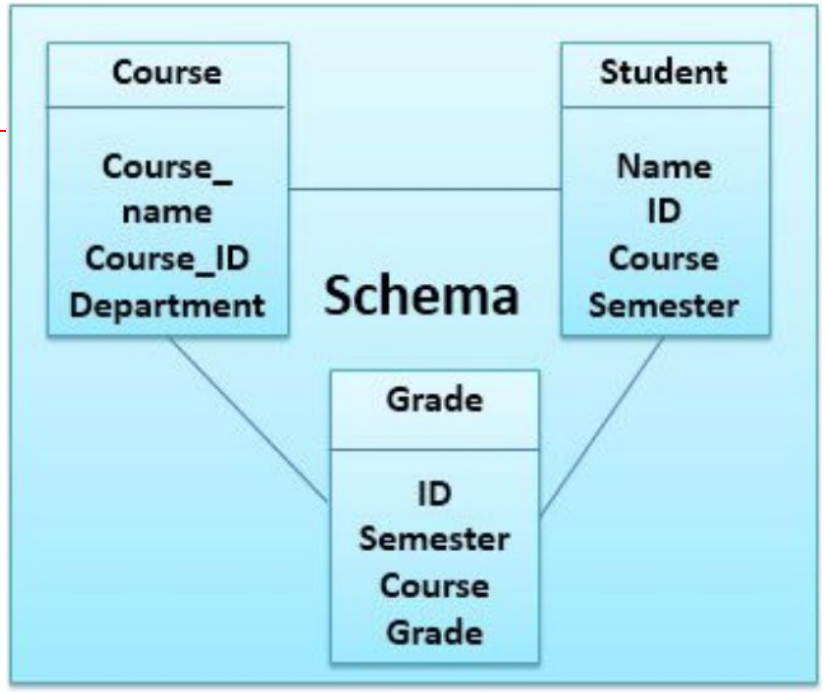
The database community has learned the following three lessons from the 40 years that have unfolded since IBM first released IMS （Information Management System) in 1968.

1) Schemas are good.
2) Separation of the schema from the application is good.
3) High-level access languages are good.

1. MapReduce is a step backwards in database access

The database community has learned the following three lessons from the 40 years that have unfolded since IBM first released IMS (Information Management System) in 1968.

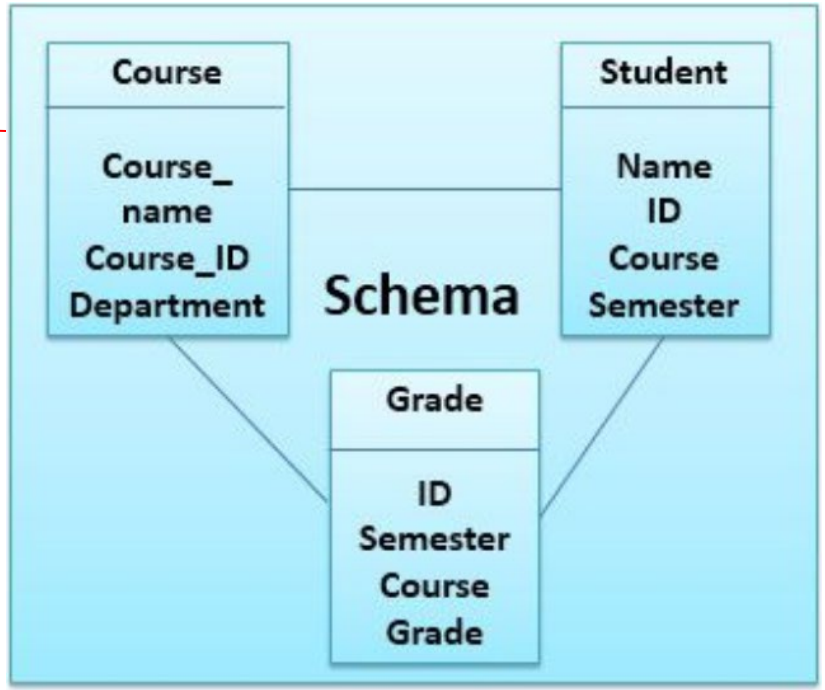
1) Schemas are good.
2) Separation of the schema from the application is good.
3) High-level access languages are good.



1. MapReduce is a step backwards in database access

The database community has learned the following three lessons from the 40 years that have unfolded since IBM first released IMS (Information Management System) in 1968.

1) Schemas are good.
2) Separation of the schema from the application is good.
3) High-level access languages are good.



MapReduce has learned none of these lessons and represents a throw back to the 1960s, before modern DBMSs were invented.

Is MapReduce a Major Step Backwards?

MapReduce may be a good idea for certain types of general-purpose computations,

But to the database community in 2008 …

1. MapReduce is a step backwards in database access
2. A sub-optimal implementation, in that it uses brute force instead of indexing
3. Not novel at all -- it represents a specific implementation of well known techniques developed nearly 25 years ago
4. Missing most of the features that are routinely included in current DBMS
5. Incompatible with all of the tools DBMS users have come to depend on

RMIT Classification: Protected

2. MapReduce is a poor implementation

All modern DBMSs use hash or B-tree indexes to accelerate access to data.

2. MapReduce is a poor implementation

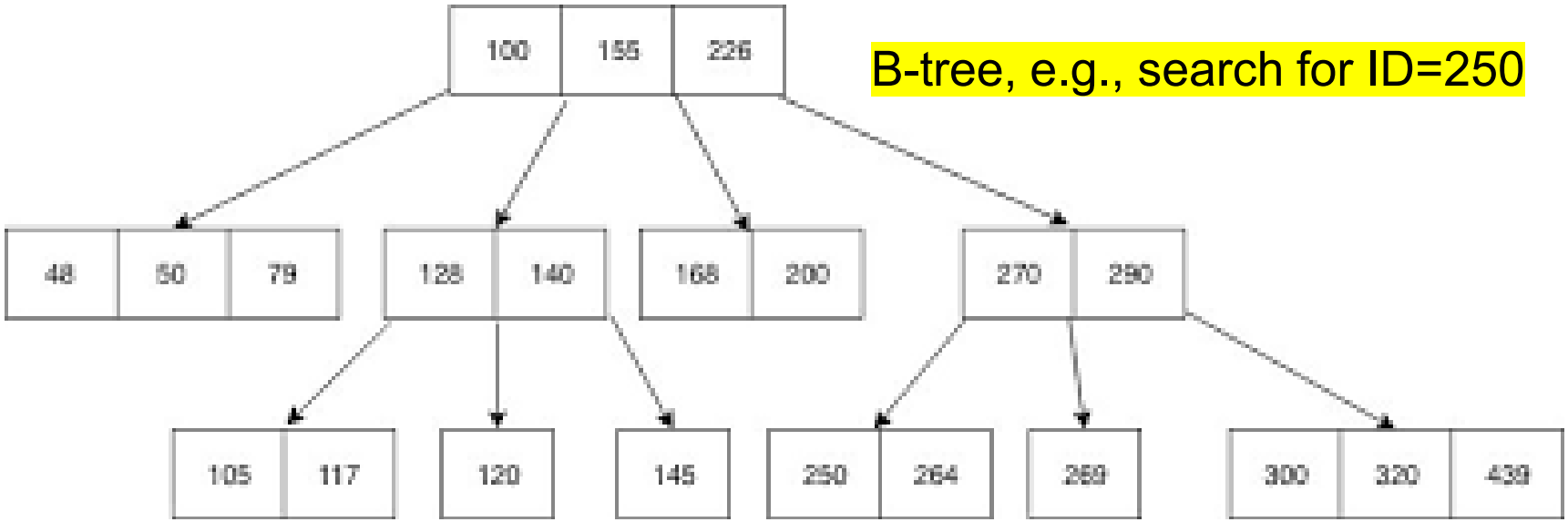
All modern DBMSs use hash or B-tree indexes to accelerate access to data.

- If one is looking for a subset of the records, then one can often use an index to reduce space by one to two orders of magnitude.
  - e.g., those employees with a salary of 10,000 or those in the shoe department

2. MapReduce is a poor implementation

All modern DBMSs use hash or B-tree indexes to accelerate access to data.

- If one is looking for a subset of the records, then one can often use an index to reduce space by one to two orders of magnitude.
  - e.g., those employees with a salary of 10,000 or those in the shoe department

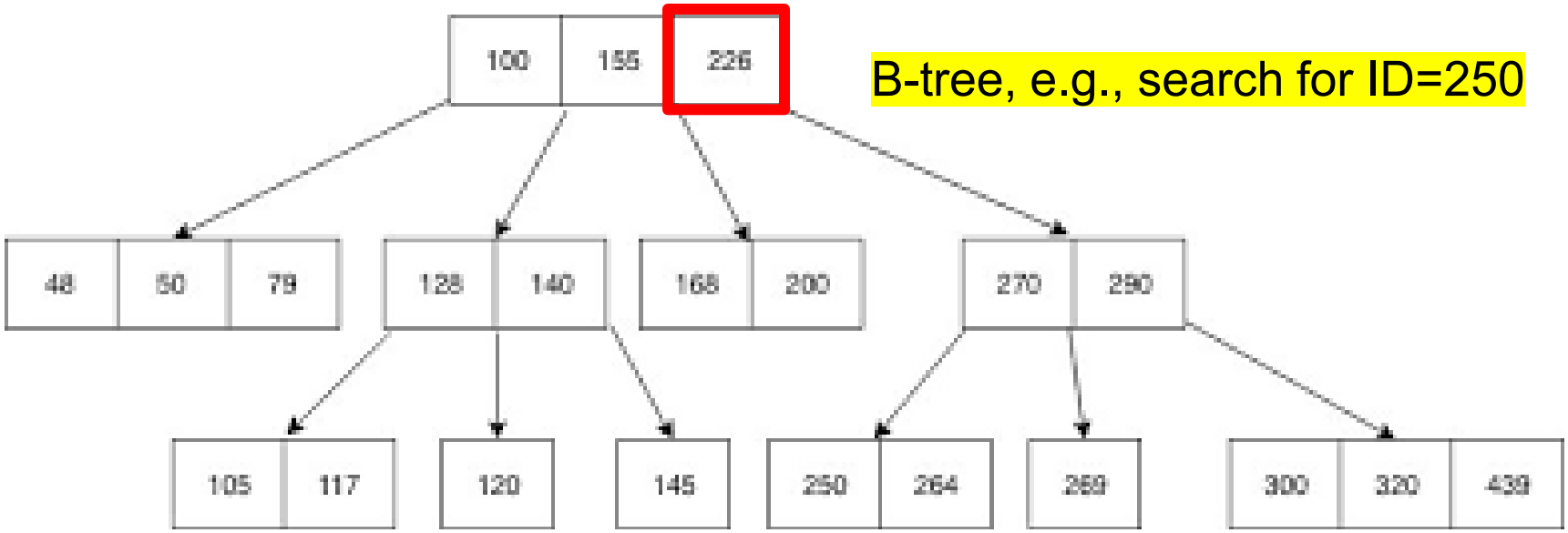


B-tree, e.g., search for ID=250

2. MapReduce is a poor implementation

All modern DBMSs use hash or B-tree indexes to accelerate access to data.

- If one is looking for a subset of the records, then one can often use an index to reduce space by one to two orders of magnitude.
  - e.g., those employees with a salary of 10,000 or those in the shoe department

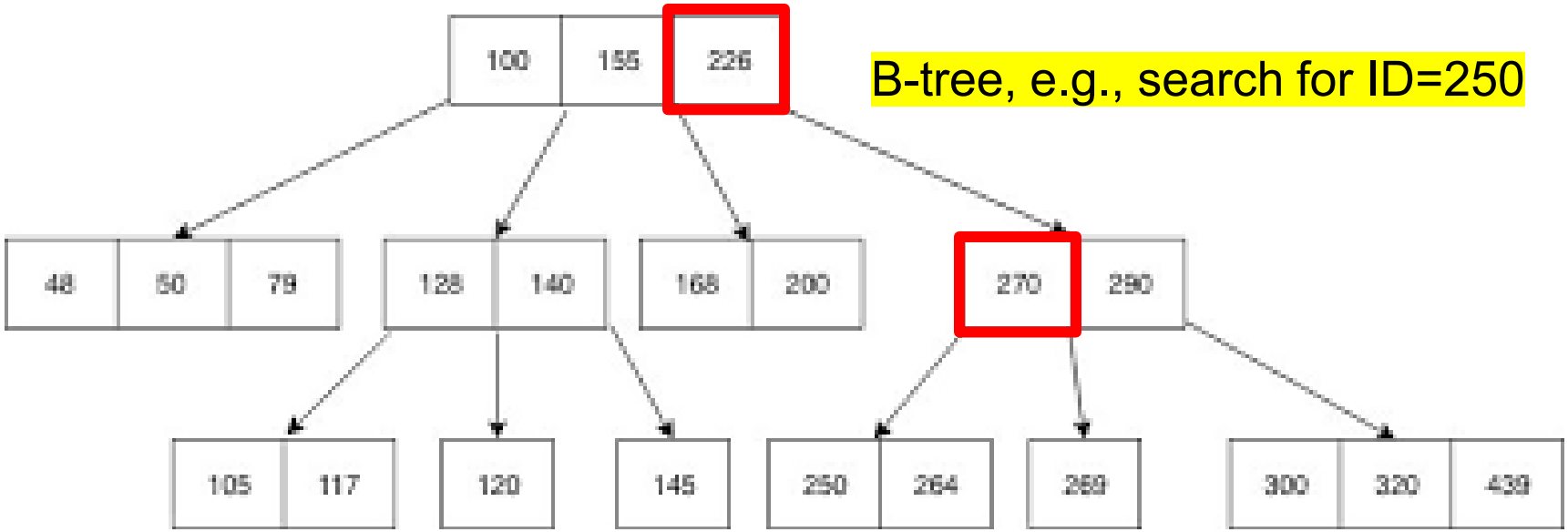


B-tree, e.g., search for ID=250

2. MapReduce is a poor implementation

All modern DBMSs use hash or B-tree indexes to accelerate access to data.

- If one is looking for a subset of the records, then one can often use an index to reduce space by one to two orders of magnitude.
  - e.g., those employees with a salary of 10,000 or those in the shoe department

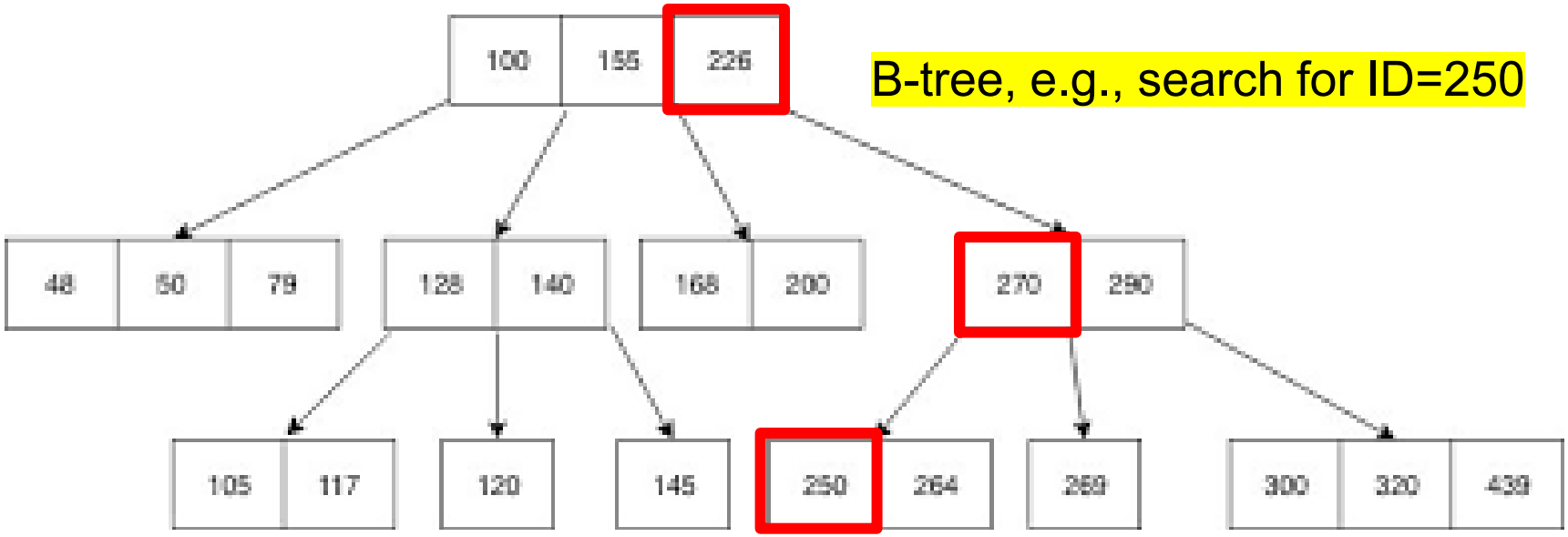


B-tree, e.g., search for ID=250

2. MapReduce is a poor implementation

All modern DBMSs use hash or B-tree indexes to accelerate access to data.

- If one is looking for a subset of the records, then one can often use an index to search space by one to two orders of magnitude.
  - e.g., those employees with a salary of 10,000 or those in the shoe department



B-tree, e.g., search for ID=250

2. MapReduce is a poor implementation

All modern DBMSs use hash or B-tree indexes to accelerate access to data.

- If one is looking for a subset of the records, then one can often use an index to search space by one to two orders of magnitude.
  - e.g., those employees with a salary of 10,000 or those in the shoe department
- A query optimizer is used to decide whether to use an index or perform a brute-force sequential search.

2. MapReduce is a poor implementation

All modern DBMSs use hash or B-tree indexes to accelerate access to data.

- If one is looking for a subset of the records, then one can often use an index to search space by one to two orders of magnitude.
  - e.g., those employees with a salary of 10,000 or those in the shoe department
- A query optimizer is used to decide whether to use an index or perform a brute-force sequential search.

MapReduce has no indexes and therefore has only brute force as a processing option. It will be beated whenever an index is the better access mechanism.

Is MapReduce a Major Step Backwards?

MapReduce may be a good idea for certain types of general-purpose computations,

But to the database community in 2008 …

1. MapReduce is a step backwards in database access
2. A sub-optimal implementation, in that it uses brute force instead of indexing
3. Not novel at all -- it represents a specific implementation of well known techniques developed nearly 25 years ago
4. Missing most of the features that are routinely included in current DBMS
5. Incompatible with all of the tools DBMS users have come to depend on

RMIT Classification: Protected

3. MapReduce is not novel

The techniques employed by MapReduce are more than 20 years old

3. MapReduce is not novel

The techniques employed by MapReduce are more than 20 years old

- The idea of partitioning a large data set into smaller partitions was first proposed in 1983 as the basis for a new type of join algorithm
  - "Application of Hash to Data Base Machine and Its Architecture," Masaru Kitsuregawa, Hidehiko Tanaka, Tohru Moto-Oka, New Generation Comput. 1(1): 63- 74 (1983)
- Cluster using a combination of partitioned tables, partitioned execution, and hash based splitting.
  - "Multiprocessor Hash-Based Join Algorithms," David J. DeWitt and Robert H. Gerber, Proceedings of the 1985 VLDB Conference, 1985.
- Execute aggregates with and without group by clauses in parallel
  - "The Gamma Database Machine Project," DeWitt, et. al., IEEE Transactions on Knowledge and Data Engineering, Vol. 2, No. 1, March 1990
- Parallel database systems and how they process queries
  - "Parallel Database System: The Future of High Performance Database Systems," David J. DeWitt and Jim Gray, CACM, Vol. 35, No. 6, June 1992.
- Alternative strategies for executing aggregates in parallel
  - "Adaptive Parallel Aggregation Algorithms," Ambuj Shatdal and Jeffrey F. Naughton, Proceedings of the 1995 SIGMOD Conference, 1995.

Is MapReduce a Major Step Backwards?

MapReduce may be a good idea for certain types of general-purpose computations,

But to the database community in 2008 …

1. MapReduce is a step backwards in database access
2. A sub-optimal implementation, in that it uses brute force instead of indexing
3. Not novel at all -- it represents a specific implementation of well known techniques developed nearly 25 years ago
4. Missing most of the features that are routinely included in current DBMS
5. Incompatible with all of the tools DBMS users have come to depend on

RMIT Classification: Protected

4. MapReduce is missing features

All of the following features are routinely provided by modern DBMSs, and all are missing from MapReduce:

4. MapReduce is missing features

All of the following features are routinely provided by modern DBMSs, and all are missing from MapReduce:

- Bulk loader -- to transform input data in files into a desired format and load it into a DBMS

4. MapReduce is missing features

All of the following features are routinely provided by modern DBMSs, and all are missing from MapReduce:

- Bulk loader -- to transform input data in files into a desired format and load it into a DBMS
- Indexing -- as noted above

4. MapReduce is missing features

All of the following features are routinely provided by modern DBMSs, and all are missing from MapReduce:

- Bulk loader -- to transform input data in files into a desired format and load it into a DBMS
- Indexing -- as noted above
- Updates -- to change the data in the database

4. MapReduce is missing features

All of the following features are routinely provided by modern DBMSs, and all are missing from MapReduce:

- Bulk loader -- to transform input data in files into a desired format and load it into a DBMS
- Indexing -- as noted above
- Updates -- to change the data in the database
- Transactions -- to support parallel update and recovery from failures during update

4. MapReduce is missing features

All of the following features are routinely provided by modern DBMSs, and all are missing from MapReduce:

- Bulk loader -- to transform input data in files into a desired format and load it into a DBMS
- Indexing -- as noted above
- Updates -- to change the data in the database
- Transactions -- to support parallel update and recovery from failures during update
- Integrity constraints -- to help keep garbage out of the database

4. MapReduce is missing features

All of the following features are routinely provided by modern DBMSs, and all are missing from MapReduce:

- Bulk loader -- to transform input data in files into a desired format and load it into a DBMS
- Indexing -- as noted above
- Updates -- to change the data in the database
- Transactions -- to support parallel update and recovery from failures during update
- Integrity constraints -- to help keep garbage out of the database



tblPerson

ID	Name	Email	GenderID
1	Jade	j@j.com	2
2	Mary	m@m.com	3
3	Martin	ma@ma.com	1
4	Rob	r@r.com	NULL
5	May	may@may.com	2
6	Kristy	k@k.com	NULL

tblGender

ID	Gender
1	Male
2	Female
3	Unknown

```
CREATE TABLE Persons (
    ID int NOT NULL,
    Name varchar(255) NOT NULL,
    Email varchar(255) NOT NULL,
    GenderID int NOT NULL
);
```

4. MapReduce is missing features

All of the following features are routinely provided by modern DBMSs, and all are missing from MapReduce:

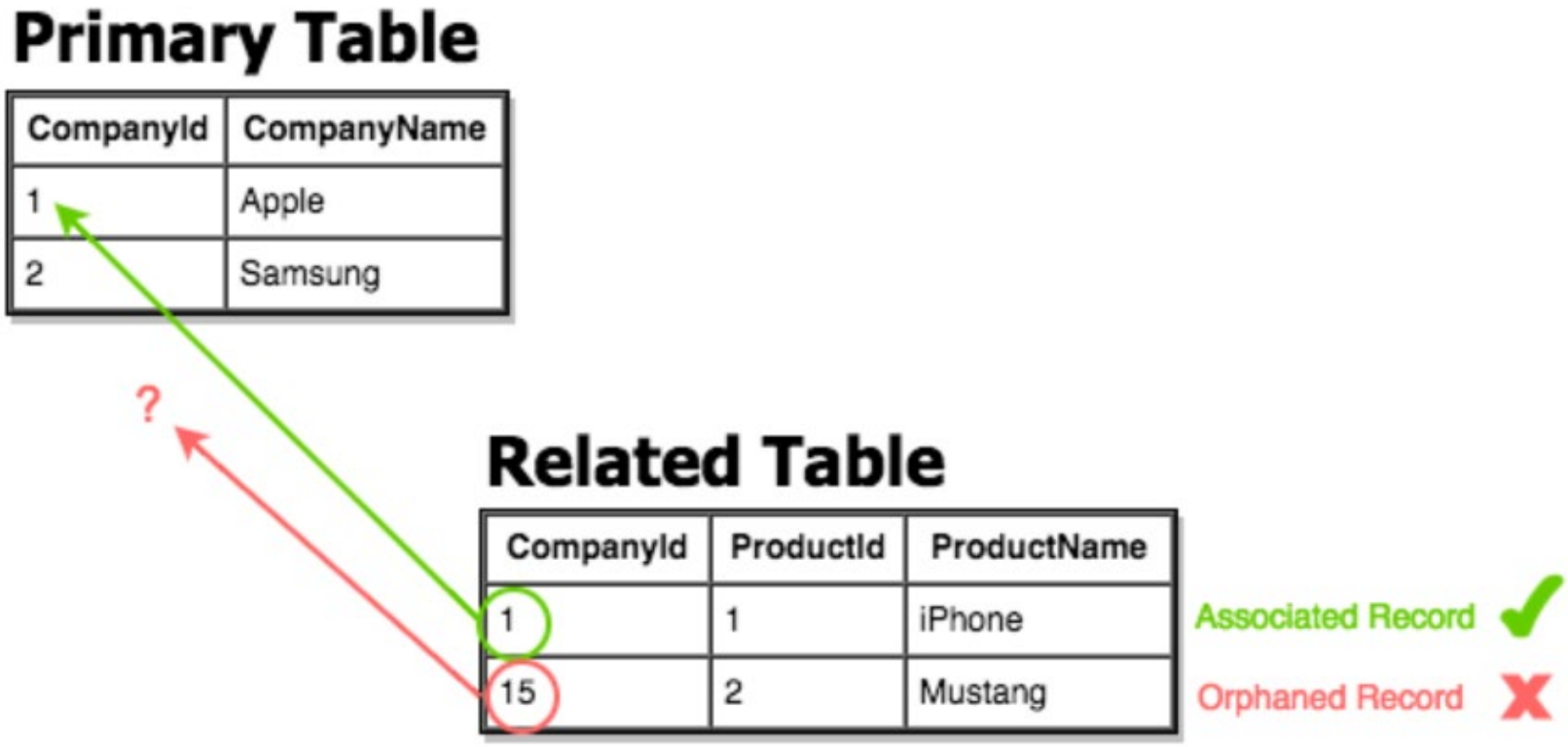
- Bulk loader -- to transform input data in files into a desired format and load it into a DBMS
- Indexing -- as noted above
- Updates -- to change the data in the database
- Transactions -- to support parallel update and recovery from failures during update
- Integrity constraints -- to help keep garbage out of the database

4. MapReduce is missing features

All of the following features are routinely provided by modern DBMSs, and all are missing from MapReduce:

- Bulk loader -- to transform input data in files into a desired format and load it into a DBMS
- Indexing -- as noted above
- Updates -- to change the data in the database
- Transactions -- to support parallel update and recovery from failures during update
- Integrity constraints -- to help keep garbage out of the database
- Referential integrity -- again, to help keep garbage out of the database

4. MapReduce is missing features



- Referential integrity -- again, to help keep garbage out of the database

4. MapReduce is missing features

All of the following features are routinely provided by modern DBMSs, and all are missing from MapReduce:

- Bulk loader -- to transform input data in files into a desired format and load it into a DBMS
- Indexing -- as noted above
- Updates -- to change the data in the database
- Transactions -- to support parallel update and recovery from failures during update
- Integrity constraints -- to help keep garbage out of the data base
- Referential integrity -- again, to help keep garbage out of the database

4. MapReduce is missing features

All of the following features are routinely provided by modern DBMSs, and all are missing from MapReduce:

- Bulk loader -- to transform input data in files into a desired format and load it into a DBMS
- Indexing -- as noted above
- Updates -- to change the data in the database
- Transactions -- to support parallel update and recovery from failures during update
- Integrity constraints -- to help keep garbage out of the data base
- Referential integrity -- again, to help keep garbage out of the database
- Views -- so the schema can change without having to rewrite the application program

Is MapReduce a Major Step Backwards?

MapReduce may be a good idea for certain types of general-purpose computations,

But to the database community in 2008 …

1. MapReduce is a step backwards in database access
2. A sub-optimal implementation, in that it uses brute force instead of indexing
3. Not novel at all -- it represents a specific implementation of well known techniques developed nearly 25 years ago
4. Missing most of the features that are routinely included in current DBMS
5. Incompatible with all of the tools DBMS users have come to depend on

RMIT Classification: Protected

5. MapReduce is incompatible with the DBMS tools

A modern SQL DBMS has available all of the following classes of tools.

RMIT Classification: Protected

5. MapReduce is incompatible with the DBMS tools

A modern SQL DBMS has available all of the following classes of tools:

- Report writers (e.g., Crystal reports) to prepare reports for human visualization

RMIT Classification: Protected

5. MapReduce is incompatible with the DBMS tools

A modern SQL DBMS has available all of the following classes of tools:

- Report writers (e.g., Crystal reports) to prepare reports for human visualization
- Business intelligence tools (e.g., Business Objects or Cognos) to enable ad-hoc querying of large data warehouses

RMIT Classification: Protected

5. MapReduce is incompatible with the DBMS tools

A modern SQL DBMS has available all of the following classes of tools:

- Report writers (e.g., Crystal reports) to prepare reports for human visualization
- Business intelligence tools (e.g., Business Objects or Cognos) to enable ad-hoc querying of large data warehouses
- Data mining tools (e.g., Oracle Data Mining or IBM DB2 Intelligent Miner) to allow a user to discover structure in large data sets

5. MapReduce is incompatible with the DBMS tools

A modern SQL DBMS has available all of the following classes of tools:

- Report writers (e.g., Crystal reports) to prepare reports for human visualization
- Business intelligence tools (e.g., Business Objects or Cognos) to enable ad-hoc querying of large data warehouses
- Data mining tools (e.g., Oracle Data Mining or IBM DB2 Intelligent Miner) to allow a user to discover structure in large data sets
- Replication tools (e.g., Golden Gate) to allow a user to replicate data from one DBMS to another

5. MapReduce is incompatible with the DBMS tools

A modern SQL DBMS has available all of the following classes of tools:

- Report writers (e.g., Crystal reports) to prepare reports for human visualization
- Business intelligence tools (e.g., Business Objects or Cognos) to enable ad-hoc querying of large data warehouses
- Data mining tools (e.g., Oracle Data Mining or IBM DB2 Intelligent Miner) to allow a user to discover structure in large data sets
- Replication tools (e.g., Golden Gate) to allow a user to replicate data from one DBMS to another
- Database design tools (e.g., Embarcadero) to assist the user in constructing a database.

RMIT Classification: Protected

5. MapReduce is incompatible with the DBMS tools

- MapReduce cannot use these tools and has none of its own.

5. MapReduce is incompatible with the DBMS tools

- MapReduce cannot use these tools and has none of its own.
- MapReduce will remain very difficult to use in an end-to-end task until
  - ✓ it becomes SQL-compatible or
  - ✓ until someone writes all of these tools

Solution - High-Level Languages

- Hadoop is great for large-data processing!
  - But writing Java/python programs for everything is verbose and slow
  - Data scientists don't always want to write Java/python

Solution - High-Level Languages

- Hadoop is great for large-data processing!
  - But writing Java/python programs for everything is verbose and slow
  - Data scientists don't always want to write Java/python
- Solution: develop higher-level data processing languages

This is the observation in 2008 ☺

Q1 - In 2008, the database community criticise "MapReduce is a step backwards". Among their arguments, which of the followings is not correct?

- MapReduce does not support transactions.
- The idea of partitioning data and processing them in parallel in MapReduce is not new.
- MapReduce does not support schemas.
- MapReduce does not support bulk load but it supports data update.
- MapReduce scan the data set rather than using index to improve the data access.

Q1 - In 2008, the database community criticise "MapReduce is a step backwards". Among their arguments, which of the followings is not correct?



MapReduce does not support transactions.

- The idea of partitioning data and processing them in parallel in MapReduce is not new.
- MapReduce does not support schemas.
- MapReduce does not support bulk load but it supports data update.
- MapReduce scan the data set rather than using index to improve the data access.

Q1 - In 2008, the database community criticise "MapReduce is a step backwards". Among their arguments, which of the followings is not correct?



MapReduce does not support transactions.



The idea of partitioning data and processing them in parallel in MapReduce is not new.

- MapReduce does not support schemas.
- MapReduce does not support bulk load but it supports data update.
- MapReduce scan the data set rather than using index to improve the data access.

Q1 - In 2008, the database community criticise "MapReduce is a step backwards". Among their arguments, which of the followings is not correct?

- ✓ MapReduce does not support transactions.
- ✓ The idea of partitioning data and processing them in parallel in MapReduce is not new.
- ✓ MapReduce does not support schemas.
- MapReduce does not support bulk load but it supports data update.
- MapReduce scan the data set rather than using index to improve the data access.

Q1 - In 2008, the database community criticise "MapReduce is a step backwards". Among their arguments, which of the followings is not correct?



MapReduce does not support transactions.



The idea of partitioning data and processing them in parallel in MapReduce is not new.



MapReduce does not support schemas.



MapReduce does not support bulk load but it supports data update.

- MapReduce scan the data set rather than using index to improve the data access.

Q1 - In 2008, the database community criticise "MapReduce is a step backwards". Among their arguments, which of the followings is not correct?

- ✓ MapReduce does not support transactions.
- ✓ The idea of partitioning data and processing them in parallel in MapReduce is not new.
- ✓ MapReduce does not support schemas.
- ✗ MapReduce does not support bulk load but it supports data update.
- ✓ MapReduce scan the data set rather than using index to improve the data access.

RMIT Classification: Protected

Q2 What is the common idea of Apache Pig and Hive?

Solution - High-Level Languages

- Hadoop is great for large-data processing!
  - But writing Java/python programs for everything is verbose and slow
  - Data scientists don't always want to write Java/python
- Solution: develop higher-level data processing languages

This is the observation in 2008 ☺

Solution - High-Level Languages

- Hadoop is great for large-data processing!
  - But writing Java/python programs for everything is verbose and slow
  - Data scientists don't always want to write Java/python
- Solution: develop higher-level data processing languages

This is the observation in 2008 ☺

Hive: HQL is like SQL

Pig: Pig Latin is a bit like Perl

Hive and Pig

- Hive: data warehousing application in Hadoop
  - Query language is HiveQL, variant of SQL
  - Tables stored on HDFS with different encodings

Developed by Facebook, now open source



Hive and Pig

- Hive: data warehousing application in Hadoop
  - Query language is HiveQL, variant of SQL
  - Tables stored on HDFS with different encodings

Developed by Facebook, now open source



Hive and Pig

- Hive: data warehousing application in Hadoop
  - Query language is HiveQL, variant of SQL
  - Tables stored on HDFS with different encodings

Developed by Facebook, now open source



Hive and Pig

- Hive: data warehousing application in Hadoop
  - Query language is HiveQL, variant of SQL
  - Tables stored on HDFS with different encodings

  Developed by Facebook, now open source



- Pig: large-scale data processing system
  - Scripts are written in Pig Latin, a dataflow language
  - Programmer focuses on data transformations

  Developed by Yahoo!, now open source



Hive and Pig

- Hive: data warehousing application in Hadoop
  - Query language is HiveQL, variant of SQL
  - Tables stored on HDFS with different encodings

  Developed by Facebook, now open source



- Pig: large-scale data processing system
  - Scripts are written in Pig Latin, a dataflow language
  - Programmer focuses on data transformations

  Developed by Yahoo!, now open source



Hive and Pig

- Hive: data warehousing application in Hadoop
  - Query language is HiveQL, variant of SQL
  - Tables stored on HDFS with different encodings

  Developed by Facebook, now open source



- Pig: large-scale data processing system
  - Scripts are written in Pig Latin, a dataflow language
  - Programmer focuses on data transformations

  Developed by Yahoo!, now open source



Hive and Pig

- Hive: data warehousing application in Hadoop
  - Query language is HiveQL, variant of SQL
  - Tables stored on HDFS with different encodings

  Developed by Facebook, now open source



- Pig: large-scale data processing system
  - Scripts are written in Pig Latin, a dataflow language
  - Programmer focuses on data transformations

  Developed by Yahoo!, now open source



Common idea:

Hive and Pig

- Hive: data warehousing application in Hadoop
  - Query language is HiveQL, variant of SQL
  - Tables stored on HDFS with different encodings

  Developed by Facebook, now open source



- Pig: large-scale data processing system
  - Scripts are written in Pig Latin, a dataflow language
  - Programmer focuses on data transformations

  Developed by Yahoo!, now open source



Common idea:

1. Provide higher-level language to facilitate large-data processing
2. Higher-level language "compiles down" to Hadoop jobs

Q2 What is the common idea of Apache Pig and Hive?

- Provide higher-level language to facilitate large-data processing
- Higher-level language “compiles down” to Hadoop jobs

Q3 - If a user has limited programming skills, will suggest PIG to him/her for big data processing? Why?

Q3 - If a user has limited programming skills, will suggest PIG to him/her for big data processing? Why?

If a user is not good at programming, she/he normally struggles to work with Hadoop, especially while performing any MapReduce tasks. Apache Pig is a choice.

Q4 - Is any task processed by PIG able to be processed by HIVE? Why?

Part II



Important characteristics of Hive

Hive is an ETL and Data warehousing tool developed on top of Hadoop Distributed File System (HDFS).

Important characteristics of Hive

Hive is an ETL and Data warehousing tool developed on top of Hadoop Distributed File System (HDFS).

ETL (extract, transform, load) is the general procedure of copying data from one or more sources into a destination system which represents the data differently from the source(s) or in a different context than the source(s).

Important characteristics of Hive

Hive is an ETL and Data warehousing tool developed on top of Hadoop Distributed File System (HDFS).

ETL (extract, transform, load) is the general procedure of copying data from one or more sources into a destination system which represents the data differently from the source(s) or in a different context than the source(s).

Date Warehouses are central repositories of integrated data from one or more disparate sources. They store current and historical data in one single place that are used for creating analytical reports for workers throughout the enterprise.

Important characteristics of Hive

Hive as data warehouse designed for managing and querying only structured data that is stored in tables.

RMIT Classification: Protected

Important characteristics of Hive

Hive as data warehouse designed for managing and querying only structured data that is stored in tables.

1. Tables and databases are created first

Important characteristics of Hive

Hive as data warehouse designed for managing and querying only structured data that is stored in tables.

1. Tables and databases are created first
   - In Hive, Metastore used for storing schema information.

Important characteristics of Hive

Hive as data warehouse designed for managing and querying only structured data that is stored in tables.

1. Tables and databases are created first
   - In Hive, Metastore used for storing schema information.
2. Then data is loaded into these tables.

Important characteristics of Hive

Hive's SQL-inspired language separates the user from the complexity of Map Reduce programming. Everyone knows it.

Select * from <TableName>

Important characteristics of Hive

Hive's SQL-inspired language separates the user from the complexity of Map Reduce programming. Everyone knows it.

Select * from <TableName>

The major difference between HQL and SQL is that Hive query executes on Hadoop's infrastructure rather than the traditional database.

Important characteristics of Hive

Hive's SQL-inspired language separates the user from the complexity of Map Reduce programming. Everyone knows it.

Select * from <TableName>

The major difference between HQL and SQL is that Hive query executes on Hadoop's infrastructure rather than the traditional database.

Hadoop's programming works on flat files. So, Hive can use directory structures to "partition" data to improve performance on certain queries.

Hive: Example

Hive looks similar to an SQL database

- Relational join on two tables:
  - Report the top most 10 frequent words from the Shakespeare collection which also appears in the Bible

Source: Material drawn from Cloudera training VM

Hive: Example

Hive looks similar to an SQL database

- Relational join on two tables:
  - Report the top 10 most frequent words from the Shakespeare collection which also appears in the Bible

```
SELECT s.word, s.freq, k.freq FROM shakespeare s
  JOIN bible k ON (s.word = k.word) WHERE s.freq >= 1 AND k.freq >= 1
  ORDER BY s.freq DESC LIMIT 10;
```

Source: Material drawn from Cloudera training VM

Hive: Example

Hive looks similar to an SQL database

- Relational join on two tables:
  - Report the top 10 most frequent words from the Shakespeare collection which also appears in the Bible

```
SELECT s.word, s.freq, k.freq FROM shakespeare s
  JOIN bible k ON (s.word = k.word) WHERE s.freq >= 1 AND k.freq >= 1
  ORDER BY s.freq DESC LIMIT 10;

the     25848   62394
I       23031   8854
and     19671   38985
to      18038   13526
of      16700   34654
a       14170   8057
you     12702   2720
my      11297   4135
in      10797   12445
is      8882    6884
```

Source: Material drawn from Cloudera training VM

```
SELECT s.word, s.freq, k.freq FROM shakespeare s
  JOIN bible k ON (s.word = k.word) WHERE s.freq >= 1 AND k.freq >= 1
  ORDER BY s.freq DESC LIMIT 10;
```

Hive:

Behind the Scenes

```
SELECT s.word, s.freq, k.freq FROM shakespeare s
  JOIN bible k ON (s.word = k.word) WHERE s.freq >= 1 AND k.freq >= 1
  ORDER BY s.freq DESC LIMIT 10;
```

Hive:
Behind the Scenes



(Abstract Syntax Tree)

```
(TOK_QUERY (TOK_FROM (TOK_JOIN (TOK_TABREF shakespeare s) (TOK_TABREF bible k) (= (. (TOK_TABLE_OR_COL s)
word) (. (TOK_TABLE_OR_COL k) word)))) (TOK_INSERT (TOK_DESTINATION (TOK_DIR TOK_TMP_FILE)) (TOK_SELECT
(TOK_SELEXPR (. (TOK_TABLE_OR_COL s) word)) (TOK_SELEXPR (. (TOK_TABLE_OR_COL s) freq)) (TOK_SELEXPR (.
(TOK_TABLE_OR_COL k) freq))) (TOK_WHERE (AND (>= (. (TOK_TABLE_OR_COL s) freq) 1) (>= (. (TOK_TABLE_OR_COL k)
freq) 1))) (TOK_ORDERBY (TOK_TABSORTCOLNAMEDESC (. (TOK_TABLE_OR_COL s) freq))) (TOK_LIMIT 10)))
```

```
SELECT s.word, s.freq, k.freq FROM shakespeare s
  JOIN bible k ON (s.word = k.word) WHERE s.freq >= 1 AND k.freq >= 1
  ORDER BY s.freq DESC LIMIT 10;
```

Hive:
Behind the Scenes



(Abstract Syntax Tree)

```
(TOK_QUERY (TOK_FROM (TOK_JOIN (TOK_TABREF shakespeare s) (TOK_TABREF bible k) (= (. (TOK_TABLE_OR_COL s)
word) (. (TOK_TABLE_OR_COL k) word)))) (TOK_INSERT (TOK_DESTINATION (TOK_DIR TOK_TMP_FILE)) (TOK_SELECT
(TOK_SELEXPR (. (TOK_TABLE_OR_COL s) word)) (TOK_SELEXPR (. (TOK_TABLE_OR_COL s) freq)) (TOK_SELEXPR (.
(TOK_TABLE_OR_COL k) freq))) (TOK_WHERE (AND (>= (. (TOK_TABLE_OR_COL s) freq) 1) (>= (. (TOK_TABLE_OR_COL k)
freq) 1))) (TOK_ORDERBY (TOK_TABSORTCOLNAMEDESC (. (TOK_TABLE_OR_COL s) freq))) (TOK_LIMIT 10)))
```



(one or more of MapReduce jobs)

Hive:
Behind the Scenes

```
STAGE DEPENDENCIES:
  Stage-1 is a root stage
  Stage-2 depends on stages: Stage-1
  Stage-0 is a root stage

STAGE PLANS:
  Stage: Stage-1
    Map Reduce
      Alias -> Map Operator Tree:
        s
          TableScan
            alias: s
            Filter Operator
              predicate:
                  expr: (freq >= 1)
                  type: boolean
              Reduce Output Operator
                key expressions:
                    expr: word
                    type: string
                sort order: +
                Map-reduce partition columns:
                    expr: word
                    type: string
                tag: 0
                value expressions:
                    expr: freq
                    type: int
                    expr: word
                    type: string
        k
          TableScan
            alias: k
            Filter Operator
              predicate:
                  expr: (freq >= 1)
                  type: boolean
              Reduce Output Operator
                key expressions:
                    expr: word
                    type: string
                sort order: +
                Map-reduce partition columns:
                    expr: word
                    type: string
                tag: 1
                value expressions:
                    expr: freq
                    type: int
      Reduce Operator Tree:
        Join Operator
          condition map:
              Inner Join 0 to 1
          condition expressions:
            0 {VALUE._col0} {VALUE._col1}
            1 {VALUE._col0}
          outputColumnNames: _col0, _col1, _col2
          Filter Operator
            predicate:
                expr: ((_col0 >= 1) and (_col2 >= 1))
                type: boolean
            Select Operator
              expressions:
                  expr: _col1
                  type: string
                  expr: _col0
                  type: int
                  expr: _col2
                  type: int
              outputColumnNames: _col0, _col1, _col2
              File Output Operator
                compressed: false
                GlobalTableId: 0
                table:
                    input format: org.apache.hadoop.mapred.SequenceFileInputFormat
                    output format: org.apache.hadoop.hive.ql.io.HiveSequenceFileOutputFormat

  Stage: Stage-2
    Map Reduce
      Alias -> Map Operator Tree:
        hdfs://localhost:8022/tmp/hive-training/364214370/10002
            Reduce Output Operator
              key expressions:
                  expr: _col1
                  type: int
              sort order: -
              tag: -1
              value expressions:
                  expr: _col0
                  type: string
                  expr: _col1
                  type: int
                  expr: _col2
                  type: int
      Reduce Operator Tree:
        Extract
          Limit
            File Output Operator
              compressed: false
              GlobalTableId: 0
              table:
                  input format: org.apache.hadoop.mapred.TextInputFormat
                  output format: org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat

  Stage: Stage-0
    Fetch Operator
      limit: 10
```

Important characteristics of Hive

We can interact with Hive using methods like

- Web GUI
- Java Database Connectivity (JDBC) interface
- Command Line Interface (CLI)
  - Most interactions tend to take place over a command line interface (CLI). Hive provides a CLI to write Hive queries using Hive Query Language(HQL)

Important characteristics of Hive

We can interact with Hive using methods like

- Web GUI
- Java Database Connectivity (JDBC) interface
- Command Line Interface (CLI)
  - Most interactions tend to take place over a command line interface (CLI). Hive provides a CLI to write Hive queries using Hive Query Language(HQL)

Important characteristics of Hive

We can interact with Hive using methods like

- Web GUI
- Java Database Connectivity (JDBC) interface
- Command Line Interface (CLI)
  - Most interactions tend to take place over a command line interface (CLI). Hive provides a CLI to write Hive queries using Hive Query Language(HQL)

Important characteristics of Hive

We can interact with Hive using methods like

- Web GUI
- Java Database Connectivity (JDBC) interface
- Command Line Interface (CLI)
  - Most interactions tend to take place over a command line interface (CLI). Hive provides a CLI to write Hive queries using Hive Query Language(HQL)

Part III
Apache Pig!



The examples in this section is from https://www.tutorialspoint.com/apache_pig/index.htm

Source: Wikipedia (Pig)

What is Apache Pig?

Programmers who are not so good at Java/python normally used to struggle working with Hadoop, especially while performing any MapReduce tasks. Apache Pig is for all such programmers.

What is Apache Pig?

Programmers who are not so good at Java/python normally used to struggle working with Hadoop, especially while performing any MapReduce tasks. Apache Pig is for all such programmers.

What is Apache Pig?

Programmers who are not so good at Java/python normally used to struggle working with Hadoop, especially while performing any MapReduce tasks. Apache Pig is for all such programmers.

What is Apache Pig?

Programmers who are not so good at Java/python normally used to struggle working with Hadoop, especially while performing any MapReduce tasks. Apache Pig is for all such programmers.

- Using **Pig Latin**, programmers can perform MapReduce tasks easily without having to type complex codes in Java/python.

What is Apache Pig?

Programmers who are not so good at Java/python normally used to struggle working with Hadoop, especially while performing any MapReduce tasks. Apache Pig is for all such programmers.

- Using **Pig Latin**, programmers can perform MapReduce tasks easily without having to type complex codes in Java/python.
- Pig Latin is **SQL-like language** and it is easy to learn Apache Pig when you are familiar with SQL.

What is Apache Pig?

Programmers who are not so good at Java normally used to struggle working with Hadoop, especially while performing any MapReduce tasks. Apache Pig is for all such programmers.

- Using **Pig Latin**, programmers can perform MapReduce tasks easily without having to type complex codes in Java.
- Pig Latin is **SQL-like language** and it is easy to learn Apache Pig when you are familiar with SQL.
- Apache Pig provides many built-in operators to support data operations
  - like joins, filters, ordering, etc.
  - nested data types like tuples, bags, and maps.

What is Apache Pig?

Programmers who are not so good at Java normally used to struggle working with Hadoop, especially while performing any MapReduce tasks. Apache Pig is for all such programmers.

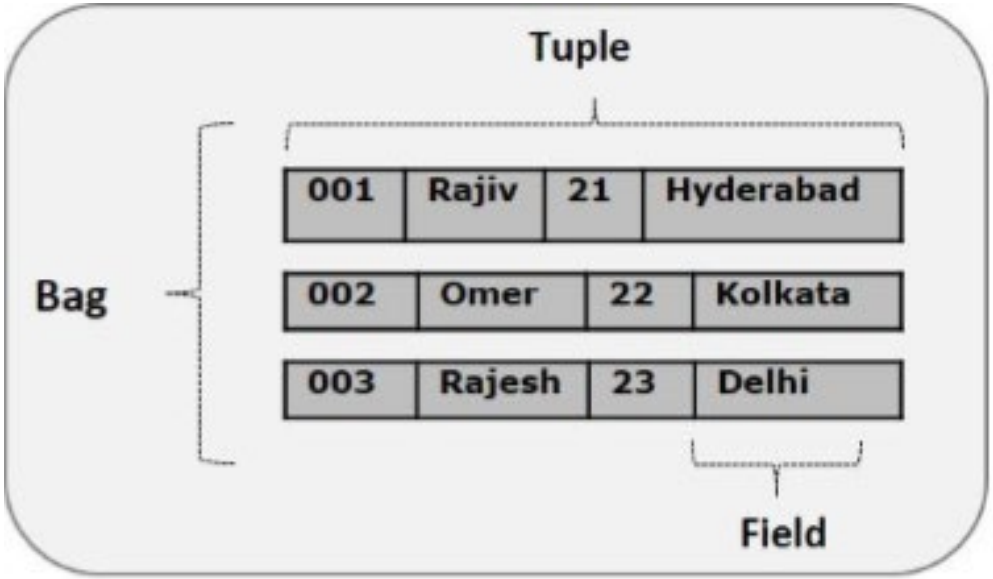
- Using **Pig Latin**, programmers can perform MapReduce tasks easily without having to type complex codes in Java.
- Pig Latin is **SQL-like language** and it is easy to learn Apache Pig when you are familiar with SQL.
- Apache Pig provides many built-in operators to support data operations
  - like joins, filters, ordering, etc.
  - nested data types like tuples, bags, and maps.

Pig Latin Data Model

The data model of Pig Latin is fully nested and it allows atomic data type **field** and complex non-atomic data types such as **map** and **tuple**.

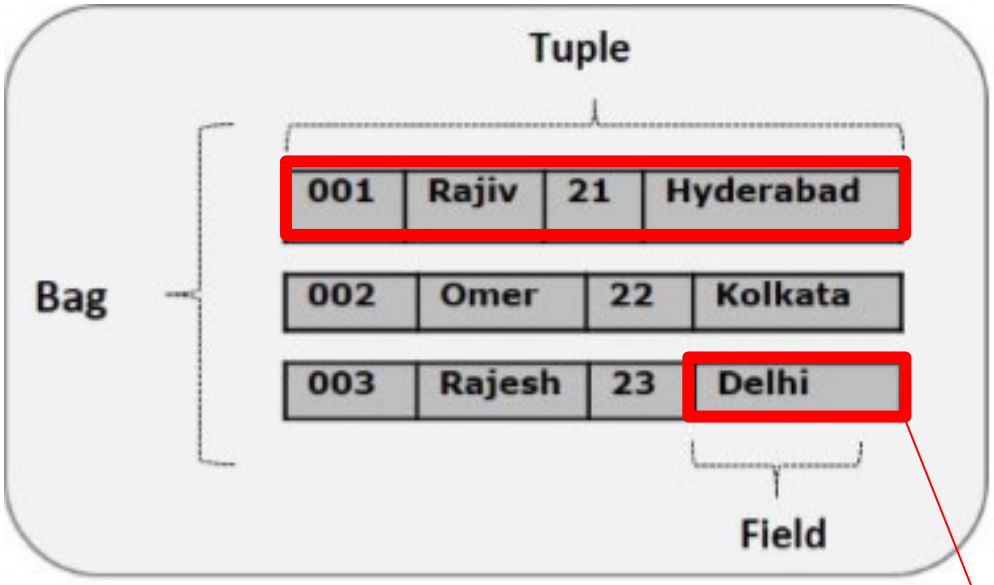
Pig Latin Data Model

The data model of Pig Latin is fully nested and it allows atomic data type **field** and complex non-atomic data types such as **map** and **tuple**.



Pig Latin Data Model

The data model of Pig Latin is fully nested and it allows atomic data type field and complex non-atomic data types such as map and tuple.

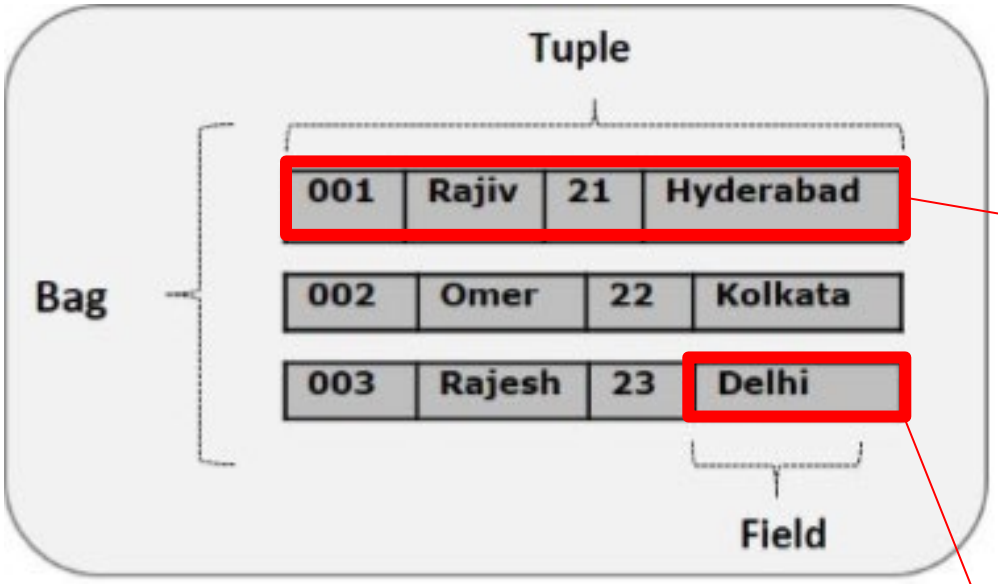


Atomic Data Type – Field - Any single value in Pig Latin.

- It is stored as string and can be used as string and number. *int, long, float, double, chararray, and bytearray* are the atomic values of Pig.
- A piece of data or a simple atomic value is known as a field.

Pig Latin Data Model

The data model of Pig Latin is fully nested and it allows atomic data type field and complex non-atomic data types such as map and tuple.



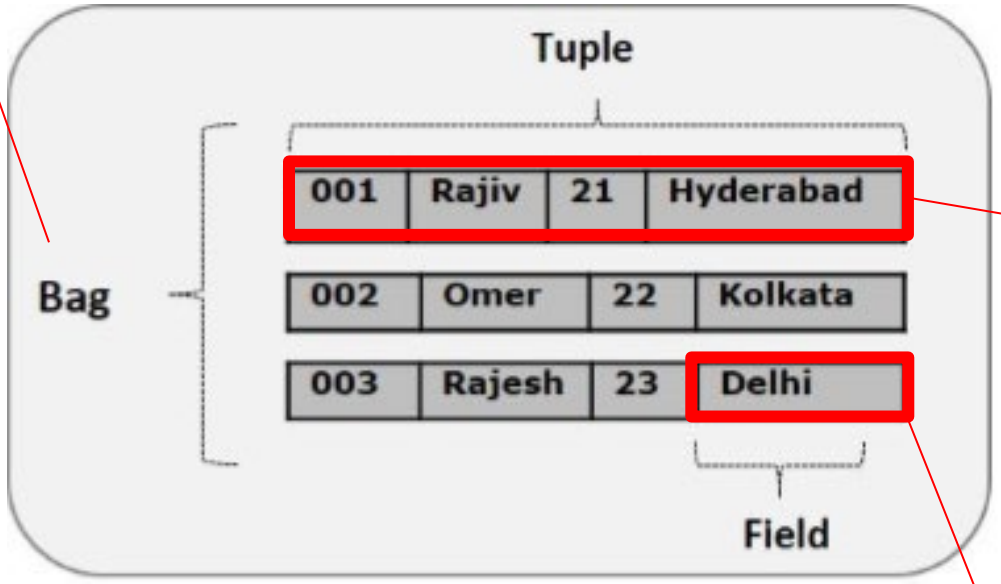
Tuple
A record that is formed by an ordered set of fields is known as a tuple, the fields can be of any type. A tuple is similar to a row in a table of RDBMS.
Example – (xx, xx, xx, xx)

Atomic Data Type – Field - Any single value in Pig Latin.

- It is stored as string and can be used as string and number. *int, long, float, double, chararray, and bytearray* are the atomic values of Pig.
- A piece of data or a simple atomic value is known as a field.

Bag - A bag represented by '{}' is an unordered set of tuples.

- Similar to a table in RDBMS, but unlike a table in RDBMS



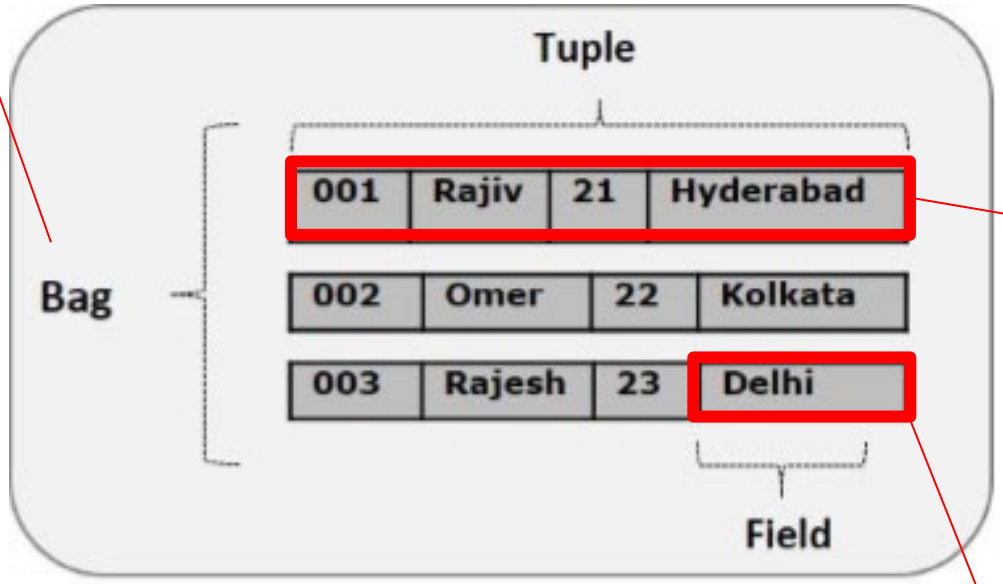
Tuple
A record that is formed by an ordered set of fields is known as a tuple, the fields can be of any type. A tuple is similar to a row in a table of RDBMS.
Example – (xx, xx, xx, xx)

Atomic Data Type – Field - Any single value in Pig Latin.

- It is stored as string and can be used as string and number. *int, long, float, double, chararray, and bytearray* are the atomic values of Pig.
- A piece of data or a simple atomic value is known as a field.

Bag - A bag represented by '{}' is an unordered set of tuples.

- Similar to a table in RDBMS, but unlike a table in RDBMS
  - it is not necessary that every tuple contain the same number of fields or that the fields in the same position (column) have the same type.
    Example – {(xxx, xxx), (xxx, xxx, xxx)}



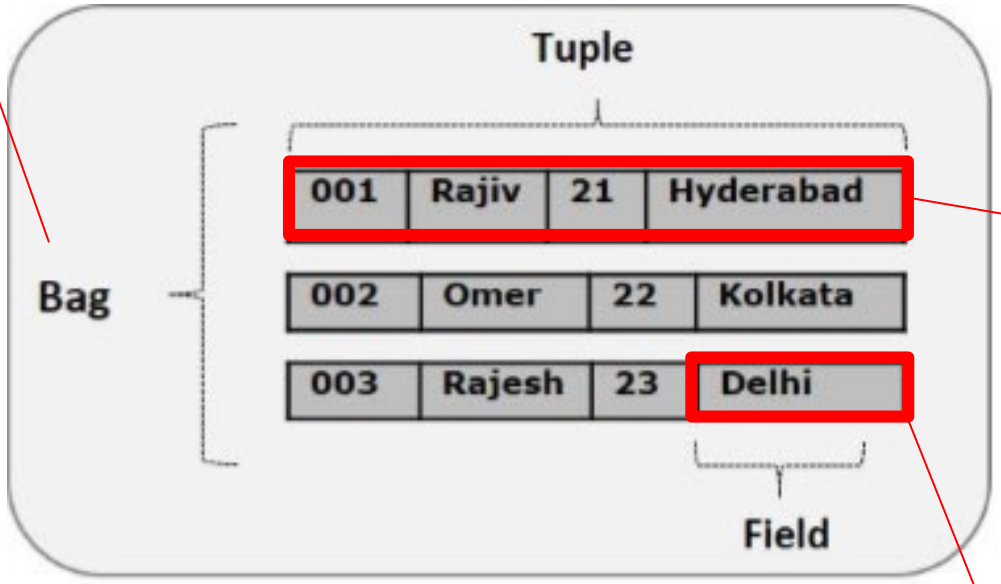
Tuple
A record that is formed by an ordered set of fields is known as a tuple, the fields can be of any type. A tuple is similar to a row in a table of RDBMS.
Example – (xx, xx, xx, xx)

Atomic Data Type – Field - Any single value in Pig Latin.

- It is stored as string and can be used as string and number. *int, long, float, double, chararray, and bytearray* are the atomic values of Pig.
- A piece of data or a simple atomic value is known as a field.

Bag - A bag represented by '{}' is an unordered set of tuples.

- Similar to a table in RDBMS, but unlike a table in RDBMS
  - it is not necessary that every tuple contain the same number of fields or that the fields in the same position (column) have the same type.
    Example – {(xxx, xxx), (xxx, xxx, xxx)}
- A bag can be a field in a relation; in that context, it is known as inner bag.
  Example – {(xxx, xxx), {(xxx, xxx, xxx)}}



Tuple
A record that is formed by an ordered set of fields is known as a tuple, the fields can be of any type. A tuple is similar to a row in a table of RDBMS.
Example – (xx, xx, xx, xx)

Atomic Data Type – Field - Any single value in Pig Latin.

- It is stored as string and can be used as string and number. *int, long, float, double, chararray, and bytearray* are the atomic values of Pig.
- A piece of data or a simple atomic value is known as a field.

Pig Latin Data Model

- **Relation**
  - A relation is a bag of tuples. The relations in Pig Latin are unordered (there is no guarantee that tuples are processed in any particular order).

Pig Latin Data Model

- **Relation**
  - A relation is a bag of tuples. The relations in Pig Latin are unordered (there is no guarantee that tuples are processed in any particular order).
- **Map**
  - A map (or data map) is a set of key-value pairs. The key needs to be of type chararray and should be unique.
  - The value might be of any type. It is represented by '[ ]'.

    [key#value <, key#value …>]

Pig Latin Data Model

- **Relation**
  - A relation is a bag of tuples. The relations in Pig Latin are unordered (there is no guarantee that tuples are processed in any particular order).
- **Map**
  - A map (or data map) is a set of key-value pairs. The key needs to be of type chararray and should be unique.
  - The value might be of any type. It is represented by '[ ]'.

    [key#value <, key#value …>]

Apache Pig - Some Common Operations

- LOAD: load data
- FOREACH … GENERATE: per tuple processing
- FILTER: discard unwanted tuples
- GROUP/COGROUP: group tuples
- JOIN: relational join

Apache Pig

LOAD

```
A = LOAD 'myfile.txt' AS (f1: int, f2: int, f3: int);
```

```
(1, 2, 3)
(4, 2, 1)
(8, 3, 4)
(4, 3, 3)
(7, 2, 5)
(8, 4, 3)
```

Apache Pig

GROUP

```
A = LOAD 'myfile.txt' AS (f1: int, f2: int, f3: int);
```

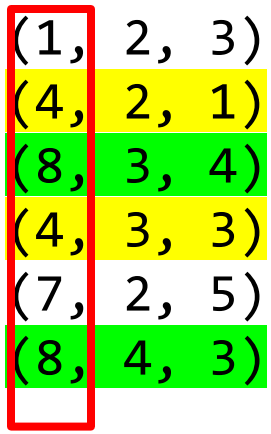
```
(1, 2, 3)
(4, 2, 1)
(8, 3, 4)
(4, 3, 3)
(7, 2, 5)
(8, 4, 3)
```

```
X = GROUP A BY f1;
```

Apache Pig

GROUP

```
A = LOAD 'myfile.txt' AS (f1: int, f2: int, f3: int);
```




```
X = GROUP A BY f1;
```

Apache Pig

GROUP

```
A = LOAD 'myfile.txt' AS (f1: int, f2: int, f3: int);
```

```
(1, 2, 3)
(4, 2, 1)
(8, 3, 4)
(4, 3, 3)
(7, 2, 5)
(8, 4, 3)
```

```
X = GROUP A BY f1;
```

```
(1, {(1, 2, 3)})
(4, {(4, 2, 1), (4, 3, 3)})
(7, {(7, 2, 5)})
(8, {(8, 3, 4), (8, 4, 3)})
```

Apache Pig
COGROUP

A:
(1, 2, 3)
(4, 2, 1)
(8, 3, 4)
(4, 3, 3)
(7, 2, 5)
(8, 4, 3)

B:
(2, 4)
(8, 9)
(1, 3)
(2, 7)
(2, 9)
(4, 6)
(4, 9)

```
X = COGROUP A BY f1, B BY $0;
```

Apache Pig
COGROUP

A:

```
(1, 2, 3)
(4, 2, 1)
(8, 3, 4)
(4, 3, 3)
(7, 2, 5)
(8, 4, 3)
```

B:

```
(2, 4)
(8, 9)
(1, 3)
(2, 7)
(2, 9)
(4, 6)
(4, 9)
```

```
X = COGROUP A BY f1, B BY $0;
```

Apache Pig
COGROUP

A:
(1, 2, 3)
(4, 2, 1)
(8, 3, 4)
(4, 3, 3)
(7, 2, 5)
(8, 4, 3)

B:
(2, 4)
(8, 9)
(1, 3)
(2, 7)
(2, 9)
(4, 6)
(4, 9)

```
X = COGROUP A BY f1, B BY $0;
```

```
(1, {(1, 2, 3)}, {(1, 3)})
(2, {}, {(2, 4), (2, 7), (2, 9)})
(4, {(4, 2, 1), (4, 3, 3)}, {(4, 6),(4, 9)})
(7, {(7, 2, 5)}, {})
(8, {(8, 3, 4), (8, 4, 3)}, {(8, 9)})
```

RMIT Classification: Protected

Apache Pig

JOIN

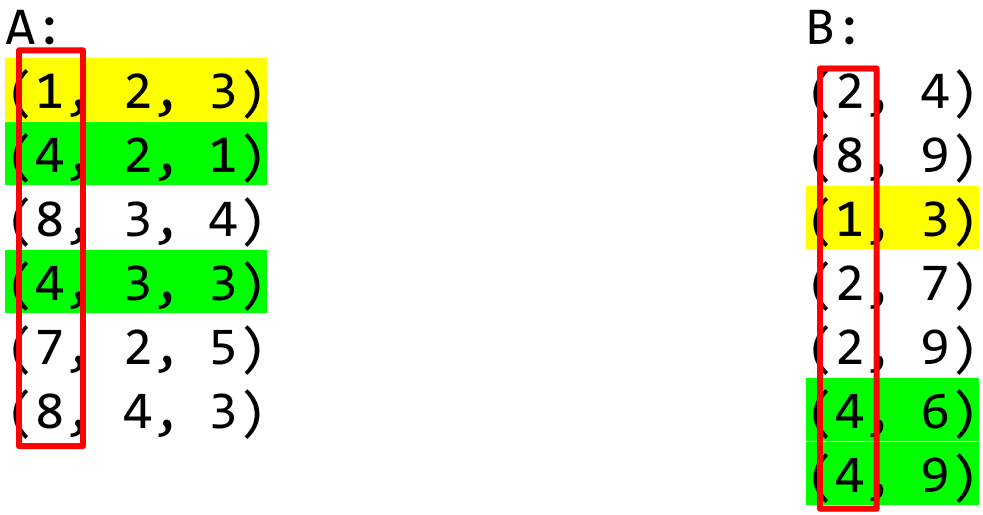
A:
(1, 2, 3)
(4, 2, 1)
(8, 3, 4)
(4, 3, 3)
(7, 2, 5)
(8, 4, 3)

B:
(2, 4)
(8, 9)
(1, 3)
(2, 7)
(2, 9)
(4, 6)
(4, 9)

```
X = JOIN A BY $0, B BY $0;
```

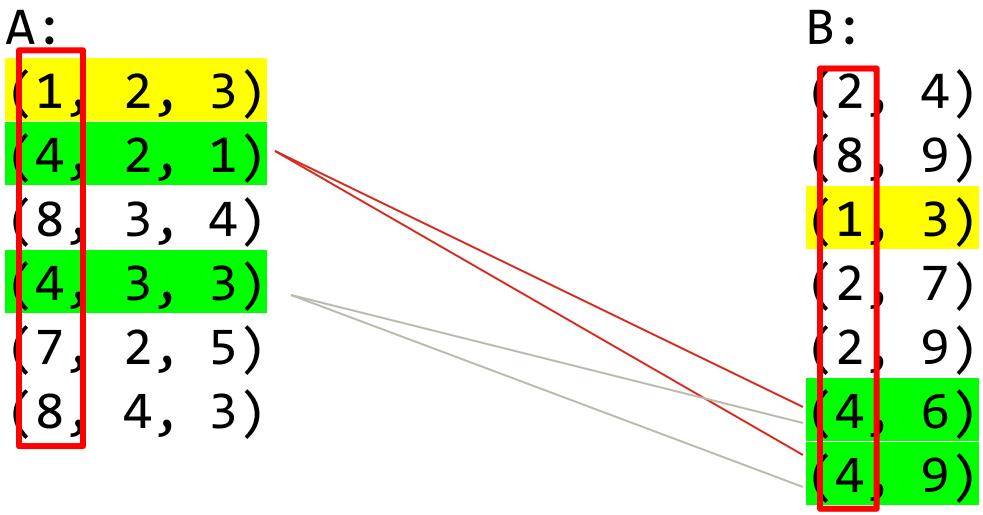
Apache Pig

JOIN




```
X = JOIN A BY $0, B BY $0;
```

Apache Pig
JOIN




```
X = JOIN A BY $0, B BY $0;
```

```
(1,2,3,1,3)
(4,2,1,4,6)
(4,3,3,4,6)
(4,2,1,4,9)
(4,3,3,4,9)
(8,3,4,8,9)
(8,4,3,8,9)
```

Apache Pig Execution Modes

You can run Apache Pig in two modes

Apache Pig Execution Modes

You can run Apache Pig in two modes

- **Local Mode**
  - In this mode, all the files are installed and run from your local host and local file system. There is no need of Hadoop or HDFS.
  - This mode is generally used for testing purpose.

Apache Pig Execution Modes

You can run Apache Pig in two modes

- **Local Mode**
  - In this mode, all the files are installed and run from your local host and local file system. There is no need of Hadoop or HDFS.
  - This mode is generally used for testing purpose.
- **MapReduce Mode**
  - MapReduce mode is where we load or process the data that exists in the Hadoop File System (HDFS) using Apache Pig.
  - In this mode, whenever we execute the Pig Latin statements to process the data, a MapReduce job is invoked in the back-end to perform a particular operation on the data that exists in the HDFS.

Apache Pig Execution Mechanisms

Apache Pig scripts can be executed in three ways

- **Interactive Mode** (Grunt shell) − You can run Apache Pig in interactive mode using the Grunt shell. In this shell, you can enter the Pig Latin statements and get the output (using Dump operator).
- **Batch Mode** (Script) − You can run Apache Pig in Batch mode by writing the Pig Latin script in a single file with **.pig** extension.
- **Embedded Mode** (UDF) − Apache Pig provides the provision of defining our own functions (**U**ser **D**efined **F**unctions) in programming languages such as Java, and using them in our script.

Apache Pig Execution Mechanisms

Interactive Mode (Grunt shell)

You can invoke the Grunt shell in a desired mode (local/MapReduce) using the **–x** option as shown below

- **Local mode**

```
$ ./pig –x local
```

- **MapReduce mode**

```
$ ./pig -x mapreduce
```

Either of these commands gives you the Grunt shell prompt as shown below.

```
grunt>
```

Apache Pig Execution Mechanisms

Interactive Mode (Grunt shell)

After invoking the Grunt shell, you can execute a Pig script by directly entering the Pig Latin statements in it.

```
grunt> student = LOAD 'hdfs://localhost:9000/pig_data/student_data.txt'
                 USING PigStorage(',')
                 as (id:int, firstname:chararray, lastname:chararray,
                     phone:chararray, city:chararray);
```

Student_data.txt

```
001,Rajiv,Reddy,9848022337,Hyderabad
002,siddarth,Battacharya,9848022338,Kolkata
003,Rajesh,Khanna,9848022339,Delhi
004,Preethi,Agarwal,9848022330,Pune
005,Trupthi,Mohanthy,9848022336,Bhuwaneshwar
006,Archana,Mishra,9848022335,Chennai.
```

Apache Pig Execution Mechanisms

Interactive Mode (Grunt shell)

After invoking the Grunt shell, you can execute a Pig script by directly entering the Pig Latin statements in it.

```
grunt> student = LOAD 'hdfs://localhost:9000/pig_data/student_data.txt'
                 USING PigStorage(',')
                 as (id:int, firstname:chararray, lastname:chararray,
                     phone:chararray, city:chararray);
```

Relation name	We have stored the data in the schema **student**.
Input file path	We are reading data from the file **student_data.txt,** which is in the /pig_data/ directory of HDFS.
Storage function	We have used the **PigStorage()** function. It loads and stores data as structured text files. It takes a delimiter using which each entity of a tuple is separated, as a parameter. By default, it takes '\t' as a parameter.

Apache Pig Execution Mechanisms

Interactive Mode (Grunt shell)

After invoking the Grunt shell, you can execute a Pig script by directly entering the Pig Latin statements in it.

```
grunt> student = LOAD 'hdfs://localhost:9000/pig_data/student_data.txt'
                 USING PigStorage(',')
                 as (id:int, firstname:chararray, lastname:chararray,
                     phone:chararray, city:chararray);
```

Relation name	We have stored the data in the schema **student**.
Input file path	We are reading data from the file **student_data.txt,** which is in the /pig_data/ directory of HDFS.
Storage function	We have used the **PigStorage()** function. It loads and stores data as structured text files. It takes a delimiter using which each entity of a tuple is separated, as a parameter. By default, it takes '\t' as a parameter.

Apache Pig Execution Mechanisms

Interactive Mode (Grunt shell)

After invoking the Grunt shell, you can execute a Pig script by directly entering the Pig Latin statements in it.

```
grunt> student = LOAD 'hdfs://localhost:9000/pig_data/student_data.txt'
                 USING PigStorage(',')
                 as (id:int, firstname:chararray, lastname:chararray,
                     phone:chararray, city:chararray);
```

Relation name	We have stored the data in the schema **student**.
Input file path	We are reading data from the file **student_data.txt,** which is in the /pig_data/ directory of HDFS.
Storage function	We have used the **PigStorage()** function. It loads and stores data as structured text files. It takes a delimiter using which each entity of a tuple is separated, as a parameter. By default, it takes '\t' as a parameter.

Apache Pig Execution Mechanisms

Interactive Mode (Grunt shell)

After invoking the Grunt shell, you can execute a Pig script by directly entering the Pig Latin statements in it.

```
grunt> student = LOAD 'hdfs://localhost:9000/pig_data/student_data.txt'
                 USING PigStorage(',')
                 as (id:int, firstname:chararray, lastname:chararray,
                     phone:chararray, city:chararray);
```

Relation name	We have stored the data in the schema **student**.
Input file path	We are reading data from the file **student_data.txt,** which is in the /pig_data/ directory of HDFS.
Storage function	We have used the **PigStorage()** function. It loads and stores data as structured text files. It takes a delimiter using which each entity of a tuple is separated, as a parameter. By default, it takes '\t' as a parameter.

Apache Pig Execution Mechanisms

Interactive Mode (Grunt shell)

After invoking the Grunt shell, you can execute a Pig script by directly entering the Pig Latin statements in it.

```
grunt> student = LOAD 'hdfs://localhost:9000/pig_data/student_data.txt'
                 USING PigStorage(',')
                 as (id:int, firstname:chararray, lastname:chararray,
                     phone:chararray, city:chararray);
```

We have stored the data using the following schema.

column	id	firstname	lastname	phone	city
datatype	int	char array	char array	char array	char array

Apache Pig Execution Mechanisms

Apache Pig scripts can be executed in three ways

- **Interactive Mode** (Grunt shell) − You can run Apache Pig in interactive mode using the Grunt shell. In this shell, you can enter the Pig Latin statements and get the output (using Dump operator).
- **Batch Mode** (Script) − You can run Apache Pig in Batch mode by writing the Pig Latin script in a single file with **.pig** extension.
- **Embedded Mode** (UDF) − Apache Pig provides the provision of defining our own functions (**U**ser **D**efined **F**unctions) in programming languages such as Java, and using them in our script.

Apache Pig Execution Mechanisms

Batch Mode (Script)

You can write an entire Pig Latin script in a file and execute it using the –x command. We have a Pig script in a file named **sample_script.pig**

```
student = LOAD 'hdfs://localhost:9000/pig_data/student_data.txt'
          USING PigStorage(',')
          as (id:int, firstname:chararray, lastname:chararray,
              phone:chararray, city:chararray)
student_order = ORDER student BY id DESC;
student_limit = LIMIT student_order 4;
Dump student_limit;
```

Student_data.txt

```
001,Rajiv,Reddy,9848022337,Hyderabad
002,siddarth,Battacharya,9848022338,Kolkata
003,Rajesh,Khanna,9848022339,Delhi
004,Preethi,Agarwal,9848022330,Pune
005,Trupthi,Mohanthy,9848022336,Bhuwaneshwar
006,Archana,Mishra,9848022335,Chennai.
```

Apache Pig Execution Mechanisms

Batch Mode (Script)

You can write an entire Pig Latin script in a file and execute it using the –x command. We have a Pig script in a file named **sample_script.pig**

```
student = LOAD 'hdfs://localhost:9000/pig_data/student_data.txt'
          USING PigStorage(',')
          as (id:int, firstname:chararray, lastname:chararray,
              phone:chararray, city:chararray)
student_order = ORDER student BY id DESC;
student_limit = LIMIT student_order 4;
Dump student_limit;
```

Only after performing the dump operation, the MapReduce job for loading the data will be carried out.

Student_data.txt

```
001,Rajiv,Reddy,9848022337,Hyderabad
002,siddarth,Battacharya,9848022338,Kolkata
003,Rajesh,Khanna,9848022339,Delhi
004,Preethi,Agarwal,9848022330,Pune
005,Trupthi,Mohanthy,9848022336,Bhuwaneshwar
006,Archana,Mishra,9848022335,Chennai.
```

Apache Pig Execution Mechanisms

Batch Mode (Script)

You can write an entire Pig Latin script in a file and execute it using the –x command. We have a Pig script in a file named **sample_script.pig**

```
student = LOAD 'hdfs://localhost:9000/pig_data/student_data.txt'
          USING PigStorage(',')
          as (id:int, firstname:chararray, lastname:chararray,
              phone:chararray, city:chararray)
student_order = ORDER student BY id DESC;
student_limit = LIMIT student_order 4;
Dump student_limit;
```

Only after performing the dump operation, the MapReduce job for loading the data will be carried out.

Student_data.txt

```
001,Rajiv,Reddy,9848022337,Hyderabad
002,siddarth,Battacharya,9848022338,Kolkata
003,Rajesh,Khanna,9848022339,Delhi
004,Preethi,Agarwal,9848022330,Pune
005,Trupthi,Mohanthy,9848022336,Bhuwaneshwar
006,Archana,Mishra,9848022335,Chennai.
```

Apache Pig Execution Mechanisms

Batch Mode (Script)

You can write an entire Pig Latin script in a file and execute it using the –x command. We have a Pig script in a file named **sample_script.pig**

```
student = LOAD 'hdfs://localhost:9000/pig_data/student_data.txt'
           USING PigStorage(',')
           as (id:int, firstname:chararray, lastname:chararray,
               phone:chararray, city:chararray)
student_order = ORDER student BY id DESC;
student_limit = LIMIT student_order 4;
Dump student_limit;
```

Now, you can execute the script in the above file as shown below.

- **Local mode**

```
$ pig -x local Sample_script.pig
```

- **MapReduce mode**

```
$ pig -x mapreduce Sample_script.pig
```

Apache Pig Execution Mechanisms

Batch Mode (Script)

We can also execute a Pig script that resides in the HDFS.

- Suppose the Pig script **sample_script.pig** is in the HDFS directory named /pig_data/. We can execute it as shown below.

```
$ pig -x mapreduce hdfs://localhost:9000/pig_data/Sample_script.pig
```

Apache Pig Execution Mechanisms

Apache Pig scripts can be executed in three ways

- **Interactive Mode** (Grunt shell) − You can run Apache Pig in interactive mode using the Grunt shell. In this shell, you can enter the Pig Latin statements and get the output (using Dump operator).
- **Batch Mode** (Script) − You can run Apache Pig in Batch mode by writing the Pig Latin script in a single file with **.pig** extension.
- **Embedded Mode** (UDF) − Apache Pig provides the provision of defining our own functions (**U**ser **D**efined **F**unctions) in programming languages such as Java, and using them in our script.

Apache Pig Execution Mechanisms

Apache Pig UDFs

In addition to the built-in functions, Apache Pig provides extensive support for **U**ser **D**efined **F**unctions (UDF). Using these UDF's, we can define our own functions and use them.

- The UDF support is provided in six programming languages,
  - Java
  - Jython
  - Python
  - JavaScript
  - Ruby
  - Groovy
- In Apache Pig, a Java repository for UDF's named Piggybank. Using Piggybank, we can access Java UDF's written by other users, and contribute our own UDF's.

Apache Pig Execution Mechanisms

Apache Pig UDFs

In addition to the built-in functions, Apache Pig provides extensive support for **U**ser **D**efined **F**unctions (UDF). Using these UDF's, we can define our own functions and use them.

- The UDF support is provided in six programming languages,
  - Java
  - Jython
  - Python
  - JavaScript
  - Ruby
  - Groovy
- In Apache Pig, a Java repository for UDF's named Piggybank. Using Piggybank, we can access Java UDF's written by other users, and contribute our own UDF's.



VS.



Apache Pig Vs Hive

Source: https://www.tutorialspoint.com

Both Apache Pig and Hive are used to create MapReduce jobs. And in some cases, Hive operates on HDFS in a similar way Apache Pig does.

Apache Pig	Hive

Apache Pig Vs Hive

Source: https://www.tutorialspoint.com

Both Apache Pig and Hive are used to create MapReduce jobs. And in some cases, Hive operates on HDFS in a similar way Apache Pig does.

Apache Pig	Hive
Apache Pig uses a language called **Pig Latin**. It was originally created at **Yahoo**.	Hive uses a language called **HiveQL**. It was originally created at **Facebook**.

Apache Pig Vs Hive

Source: https://www.tutorialspoint.com

Both Apache Pig and Hive are used to create MapReduce jobs. And in some cases, Hive operates on HDFS in a similar way Apache Pig does.

Apache Pig	Hive
Apache Pig uses a language called **Pig Latin**. It was originally created at **Yahoo**.	Hive uses a language called **HiveQL**. It was originally created at **Facebook**.
Pig Latin is a data flow language.	HiveQL is a query processing language.

Apache Pig Vs Hive

Source: https://www.tutorialspoint.com

Both Apache Pig and Hive are used to create MapReduce jobs. And in some cases, Hive operates on HDFS in a similar way Apache Pig does.

Apache Pig	Hive
Apache Pig uses a language called **Pig Latin**. It was originally created at **Yahoo**.	Hive uses a language called **HiveQL**. It was originally created at **Facebook**.
Pig Latin is a data flow language.	HiveQL is a query processing language.
Pig Latin is a procedural language and it fits in pipeline paradigm.	HiveQL is a declarative language.
.	

Apache Pig Vs Hive

Source: https://www.tutorialspoint.com

Both Apache Pig and Hive are used to create MapReduce jobs. And in some cases, Hive operates on HDFS in a similar way Apache Pig does.

Apache Pig	Hive
Apache Pig uses a language called **Pig Latin**. It was originally created at **Yahoo**.	Hive uses a language called **HiveQL**. It was originally created at **Facebook**.
Pig Latin is a data flow language.	HiveQL is a query processing language.
Pig Latin is a procedural language and it fits in pipeline paradigm.	HiveQL is a declarative language.
Apache Pig can handle structured, unstructured, and semi-structured data.	Hive is mostly for structured data.

Q4 - Is any task processed by PIG able to be processed by HIVE? Why?

Apache PIG can handle structured, unstructed, and semi-structured data; on the other hand, HIVE is for processing structured data.

Q5 - Given the following pig script, which statements are correct?

```
visits = load 'hdfs:///user/ke/visits' using PigStorage(',') as (user, url, time);
gVisits = group visits by url;
visitCounts = foreach gVisits generate group as url, COUNT(visits) as count;
urlInfo = load 'hdfs:///user/ke/urlInfo' using PigStorage(',') as (url, category, pRank);
joinInfo = join visitCounts by url, urlInfo by url;
gCategories = group joinInfo by category;
topUrls = foreach gCategories generate TOP(2,1,joinInfo);
store topUrls into 'hdfs:///user/ke/topUrls';
```

- TOP(2,1,joinInfo) - the two tuples in relation "joinInfo" with the highest value in the first column.
- Line 8 stores 10 tuples back to disk.
- Line 4 loads a data set where the default delimiter is applied.
- Line 2 groups the loaded tuples and outputs one tuple of visits for each url.
- Line 7 outputs 1 or 2 tuples of joinInfo for each category.

Pig: Example

Task: Find the top 10 most visited pages in each category

Visits

User	URL	Time
Amy	cnn.com	8:00
Amy	bbc.com	10:00
Amy	flickr.com	10:05
Fred	cnn.com	12:00

URL Info

URL	Category	PageRank
cnn.com	News	0.9
bbc.com	News	0.8
flickr.com	Photos	0.7
espn.com	Sports	0.9

Pig Slides adapted from Olston et al. (SIGMOD 2008)

Pig: Example

Task: Find the top 10 most visited pages in each category

- Pig Script

```
1.  visits = load '/data/visits' as (user, url, time);
2.  gVisits = group visits by url;
3.  visitCounts = foreach gVisits generate url, count(visits);
4.  urlInfo = load '/data/urlInfo' as (url, category, pRank);
5.  visitCounts = join visitCounts by url, urlInfo by url;
6.  gCategories = group visitCounts by category;
7.  topUrls = foreach gCategories generate top(count,10);
8.  store topUrls into '/data/topUrls';
```

Pig Slides adapted from Olston et al. (SIGMOD 2008)

Pig: Example

Task: Find the top 10 most visited pages in each category

- Pig Script

```
1.  visits = load '/data/visits' as (user, url, time);
2.  gVisits = group visits by url;
```

Visits

User	URL	Time
Amy	cnn.com	8:00
Amy	bbc.com	10:00
Amy	flickr.com	10:05
Fred	cnn.com	12:00



gVisits

(cnn.com, {(Amy, cnn.com, 8:00), (Fred, cnn.com, 12:00)}
(bbc.com, {(Amy, bbc.com, 10:00)})
(flickr.com, {(Army, flickr.com, 10:05)})

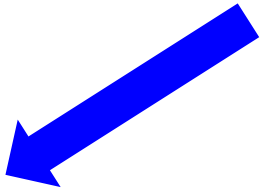
Pig: Example

Task: Find the top 10 most visited pages in each category

- Pig Script

```
1.  visits = load '/data/visits' as (user, url, time);
2.  gVisits = group visits by url;
3.  visitCounts = foreach gVisits generate url, count(visits);
```

(cnn.com, {(Amy, cnn.com, 8:00), (Fred, cnn.com, 12:00)}
(bbc.com, {(Amy, bbc.com, 10:00)})
(flickr.com, {(Army, flickr.com, 10:05)})



Pig Slides adapted from Olston et al. (SIGMOD 2008)

Pig: Example

Task: Find the top 10 most visited pages in each category

- Pig Script

```
1.  visits = load '/data/visits' as (user, url, time);
2.  gVisits = group visits by url;
3.  visitCounts = foreach gVisits generate url, count(visits);
```

(cnn.com, {(Amy, cnn.com, 8:00), (Fred, cnn.com, 12:00)}
(bbc.com, {(Amy, bbc.com, 10:00)})
(flickr.com, {(Army, flickr.com, 10:05)})

visitCounts

URL	count
cnn.com	2
bbc.com	1
flick.com	1

Pig: Example

Task: Find the top 10 most visited pages in each category

- Pig Script

```
1.  visits = load '/data/visits' as (user, url, time);
2.  gVisits = group visits by url;
3.  visitCounts = foreach gVisits generate url, count(visits);
4.  urlInfo = load '/data/urlInfo' as (url, category, pRank);
```

visitCounts

URL	count
cnn.com	2
bbc.com	1
flick.com	1

URL Info

URL	Category	PageRank
cnn.com	News	0.9
bbc.com	News	0.8
flickr.com	Photos	0.7
espn.com	Sports	0.9

Pig: Example

Task: Find the top 10 most visited pages in each category

- Pig Script

```
1.  visits = load '/data/visits' as (user, url, time);
2.  gVisits = group visits by url;
3.  visitCounts = foreach gVisits generate url, count(visits);
4.  urlInfo = load '/data/urlInfo' as (url, category, pRank);
5.  visitCounts = join visitCounts by url, urlInfo by url;
```

visitCounts

URL	count
cnn.com	2
bbc.com	1
flick.com	1

URL Info

URL	Category	PageRank
cnn.com	News	0.9
bbc.com	News	0.8
flickr.com	Photos	0.7
espn.com	Sports	0.9

Pig: Example

Task: Find the top 10 most visited pages in each category

visitCounts

URL	Category	count	PageRank
Cnn.com	News	2	0.9
Bbc.com	News	1	0.8
Flick.com	Photos	1	0.7

```
4.  urlInfo = load '/data/urlInfo' as (url, category, pRank);
5.  visitCounts = join visitCounts by url, urlInfo by url;
```

visitCounts

URL	count
cnn.com	2
bbc.com	1
flick.com	1

URL Info

URL	Category	PageRank
cnn.com	News	0.9
bbc.com	News	0.8
flickr.com	Photos	0.7
espn.com	Sports	0.9

Pig: Example

Task: Find the top 10 most visited pages in each category

visitCounts

URL	Category	count	PageRank
Cnn.com	News	2	0.9
Bbc.com	News	1	0.8
Flick.com	Photos	1	0.7

```
4.  urlInfo = load '/data/urlInfo' as (url, category, pRank);
5.  visitCounts = join visitCounts by url, urlInfo by url;
6.  gCategories = group visitCounts by category;
7.  topUrls = foreach gCategories generate top(visitCounts,10);
8.  store topUrls into '/data/topUrls';
```

Pig Slides adapted from Olston et al. (SIGMOD 2008)

Pig: Example

Task: Find the top 10 most visited pages in each category

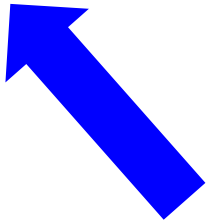
visitCounts

URL	Category	count	PageRank
Cnn.com	News	2	0.9
Bbc.com	News	1	0.8
Flick.com	Photos	1	0.7

```
4.  urlInfo = load '/data/urlInfo' as (url, category, pRank);
5.  visitCounts = join visitCounts by url, urlInfo by url;
6.  gCategories = group visitCounts by category;
7.  topUrls = foreach gCategories generate top(visitCounts,10);
8.  store topUrls into '/data/topUrls';
```

Pig Slides adapted from Olston et al. (SIGMOD 2008)

(News, {(Cnn.com,News,2,0.9) , (Bbc.com,News,1,0.8)})
(Photos, {(Flick.com,Photos,1,0.7)})



visitCounts

URL	Category	count	PageRank
Cnn.com	News	2	0.9
Bbc.com	News	1	0.8
Flick.com	Photos	1	0.7

```
4.  urlInfo = load '/data/urlInfo' as (url, category, pRank);
5.  visitCounts = join visitCounts by url, urlInfo by url;
6.  gCategories = group visitCounts by category;
7.  topUrls = foreach gCategories generate top(visitCounts,10);
8.  store topUrls into '/data/topUrls';
```

Pig Slides adapted from Olston et al. (SIGMOD 2008)

(News, {(Cnn.com,News,2,0.9) , (Bbc.com,News,1,0.8)})
(Photos, {(Flick.com,Photos,1,0.7)})

URL	Category	count	PageRank
Cnn.com	News	2	0.9
Bbc.com	News	1	0.8
Flick.com	Photos	1	0.7

```
4.  urlInfo = load '/data/urlInfo' as (url, category, pRank);
5.  visitCounts = join visitCounts by url, urlInfo by url;
6.  gCategories = group visitCounts by category;
7.  topUrls = foreach gCategories generate top(visitCounts,10);
8.  store topUrls into '/data/topUrls';
```

Pig Slides adapted from Olston et al. (SIGMOD 2008)

(News, {(Cnn.com,News,2,0.9) , (Bbc.com,News,1,0.8)})
(Photos, {(Flick.com,Photos,1,0.7)})

URL	Category	count	PageRank
Cnn.com	News	2	0.9
Bbc.com	News	1	0.8
Flick.com	Photos	1	0.7

```
4.  urlInfo = load '/data/urlInfo' as (url, category, pRank);
5.  visitCounts = join visitCounts by url, urlInfo by url;
6.  gCategories = group visitCounts by category;
7.  topUrls = foreach gCategories generate top(visitCounts,10);
8.  store topUrls into '/data/topUrls';
```

Pig Slides adapted from Olston et al. (SIGMOD 2008)

Pig: Example

Task: Find the top 10 most visited pages in each category

- Pig Script

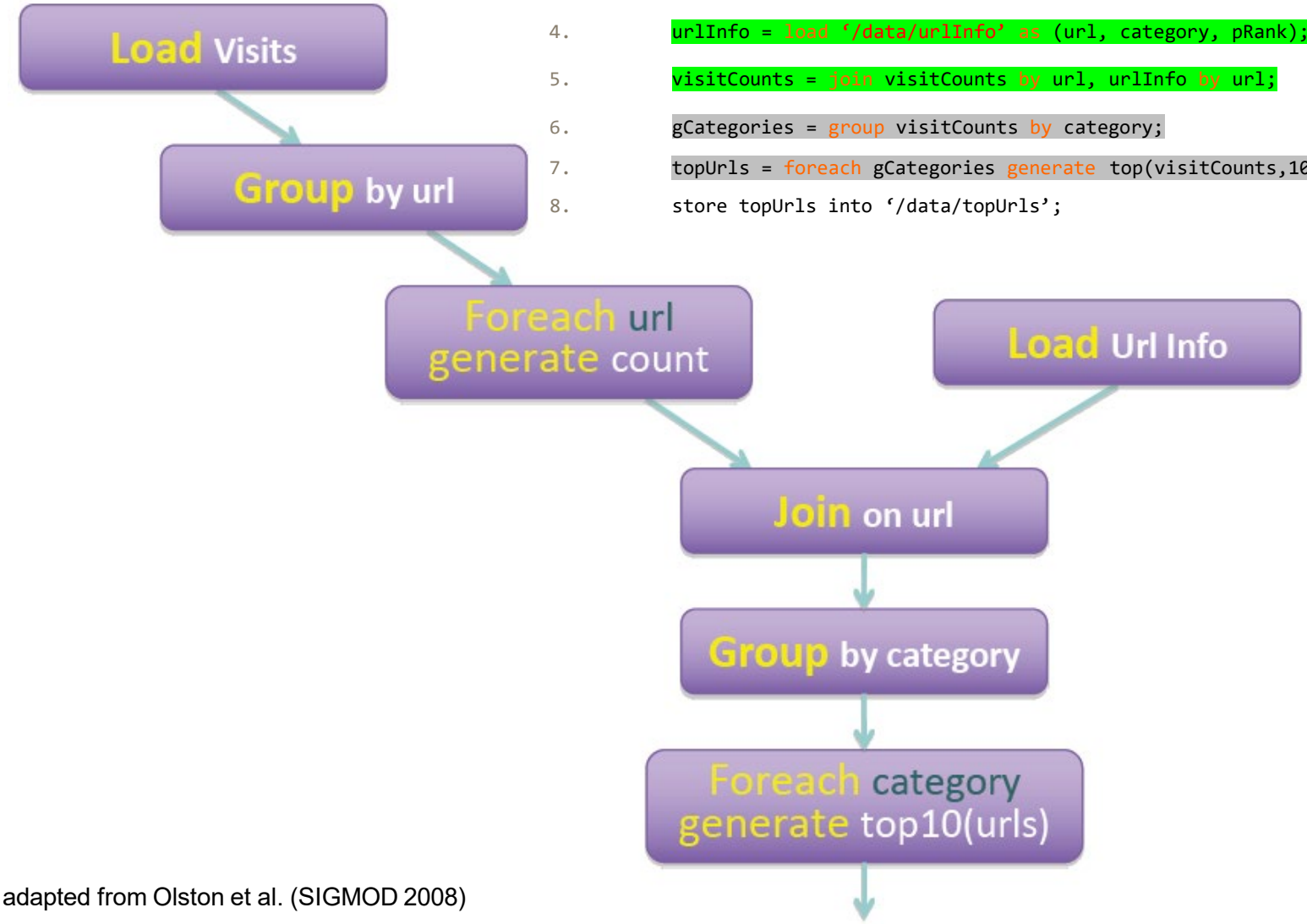
```
1.  visits = load '/data/visits' as (user, url, time);
2.  gVisits = group visits by url;
3.  visitCounts = foreach gVisits generate url, count(visits);
4.  urlInfo = load '/data/urlInfo' as (url, category, pRank);
5.  visitCounts = join visitCounts by url, urlInfo by url;
6.  gCategories = group visitCounts by category;
7.  topUrls = foreach gCategories generate top(visitCounts,10);
8.  store topUrls into '/data/topUrls';
```

Pig Slides adapted from Olston et al. (SIGMOD 2008)

Pig Query Plan

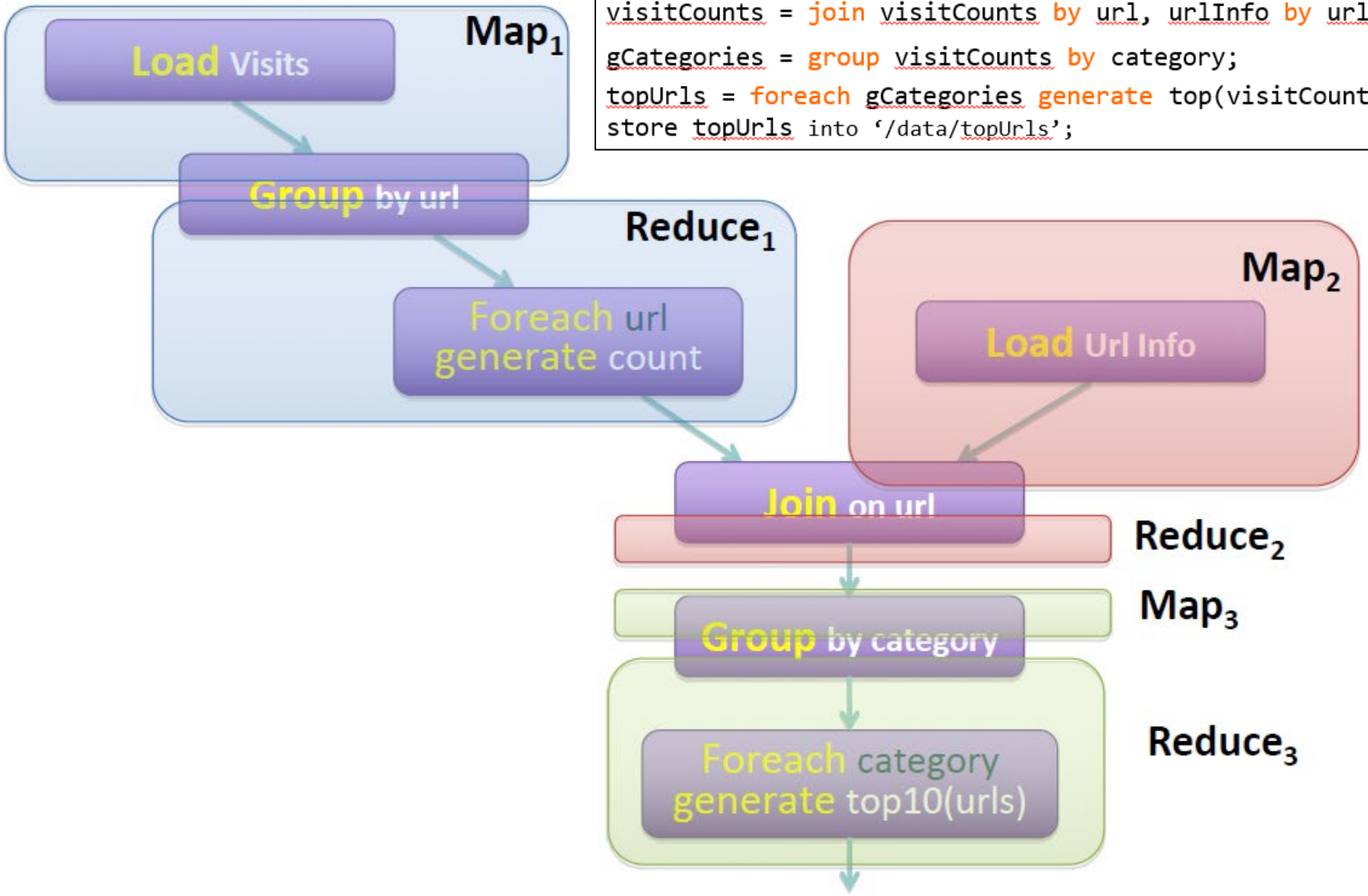
- Pig Script

```
1.  visits = load '/data/visits' as (user, url, time);
2.  gVisits = group visits by url;
3.  visitCounts = foreach gVisits generate url, count(visits);
4.  urlInfo = load '/data/urlInfo' as (url, category, pRank);
5.  visitCounts = join visitCounts by url, urlInfo by url;
6.  gCategories = group visitCounts by category;
7.  topUrls = foreach gCategories generate top(visitCounts,10);
8.  store topUrls into '/data/topUrls';
```

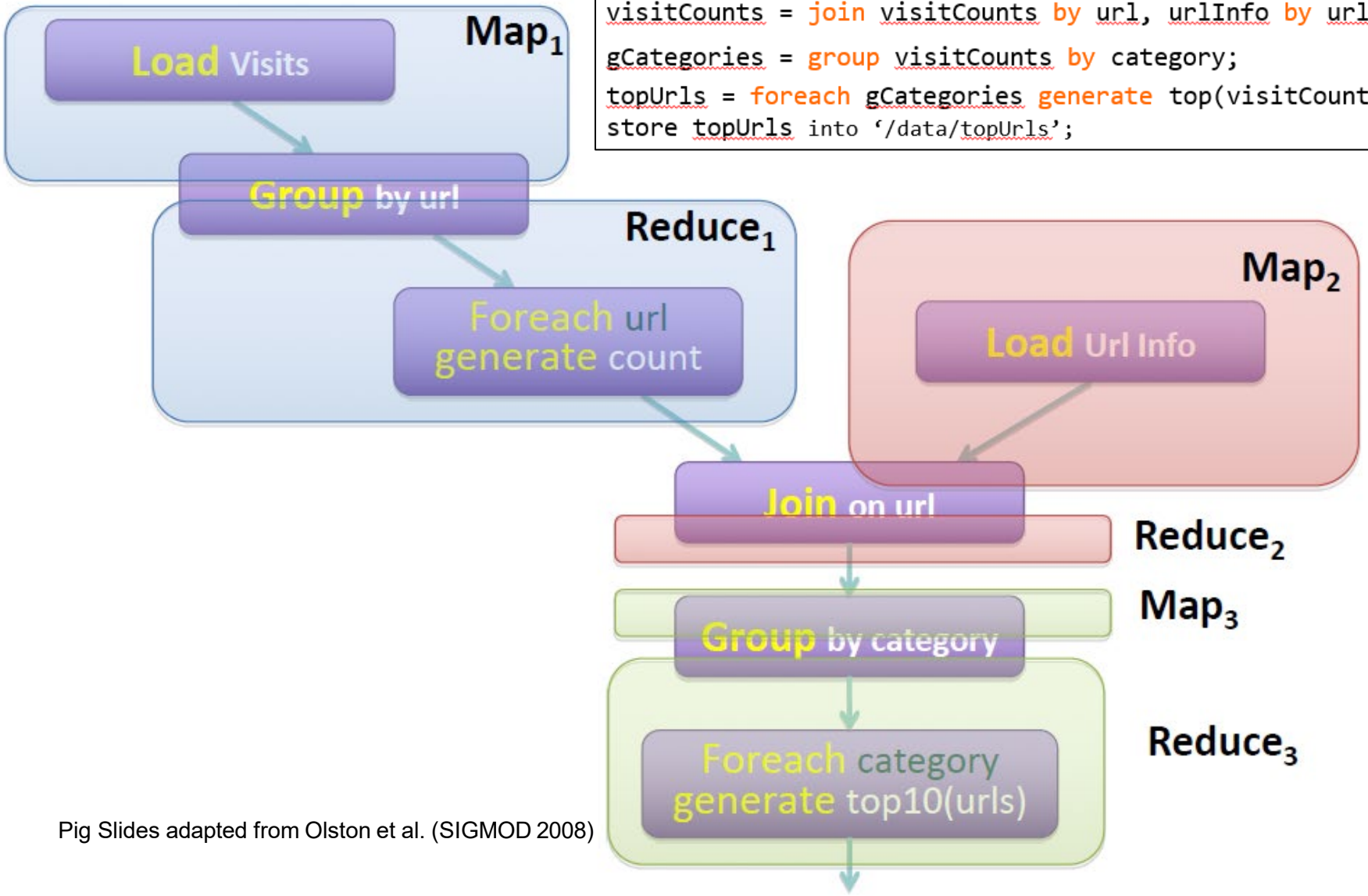


Pig Slides adapted from Olston et al. (SIGMOD 2008)

```
visits = load '/data/visits' as (user, url, time);
gVisits = group visits by url;
visitCounts = foreach gVisits generate url, count(visits);
urlInfo = load '/data/urlInfo' as (url, category, pRank);
visitCounts = join visitCounts by url, urlInfo by url;
gCategories = group visitCounts by category;
topUrls = foreach gCategories generate top(visitCounts,10);
store topUrls into '/data/topUrls';
```

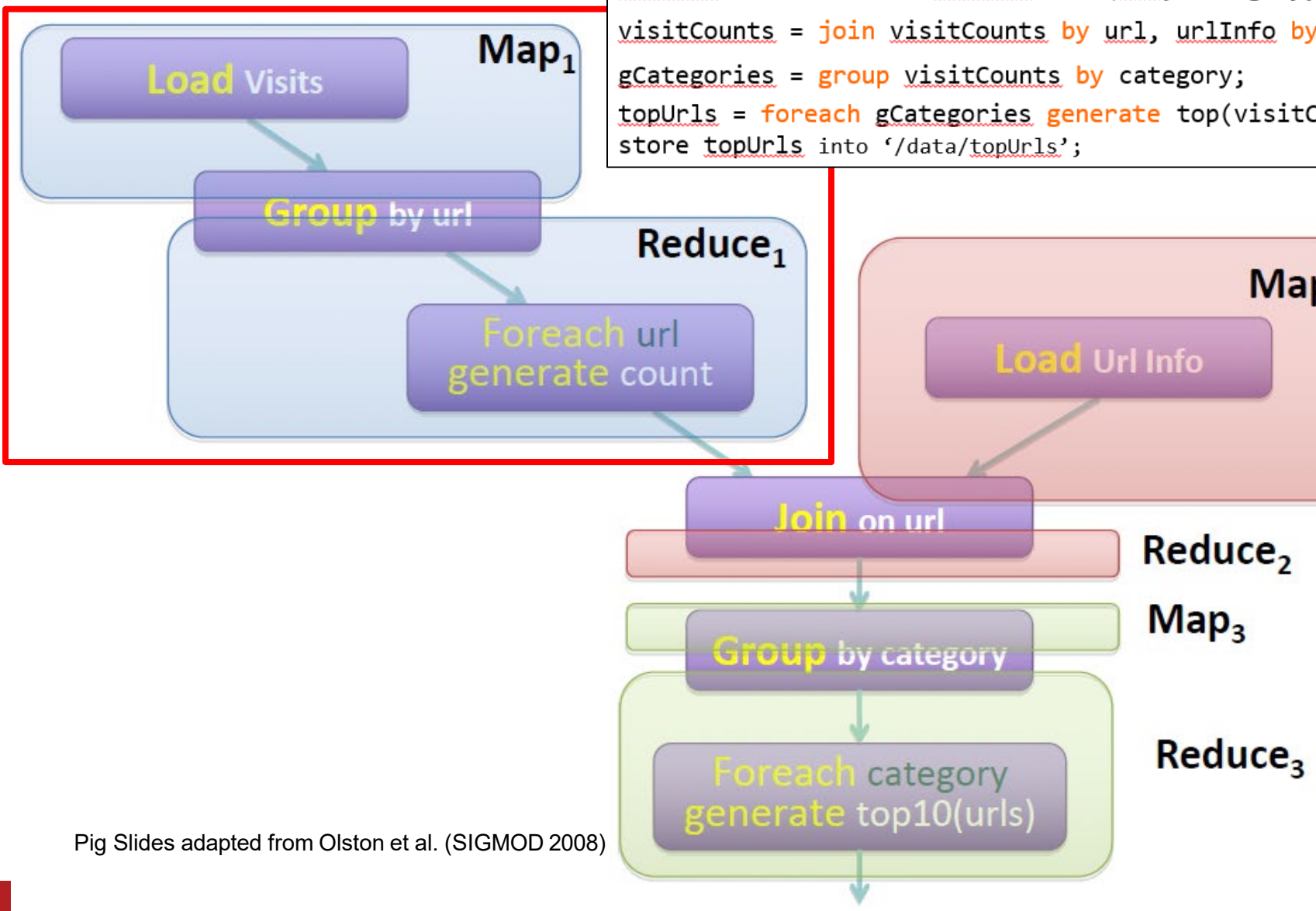


```
visits = load '/data/visits' as (user, url, time);
gVisits = group visits by url;
visitCounts = foreach gVisits generate url, count(visits);
urlInfo = load '/data/urlInfo' as (url, category, pRank);
visitCounts = join visitCounts by url, urlInfo by url;
gCategories = group visitCounts by category;
topUrls = foreach gCategories generate top(visitCounts,10);
store topUrls into '/data/topUrls';
```



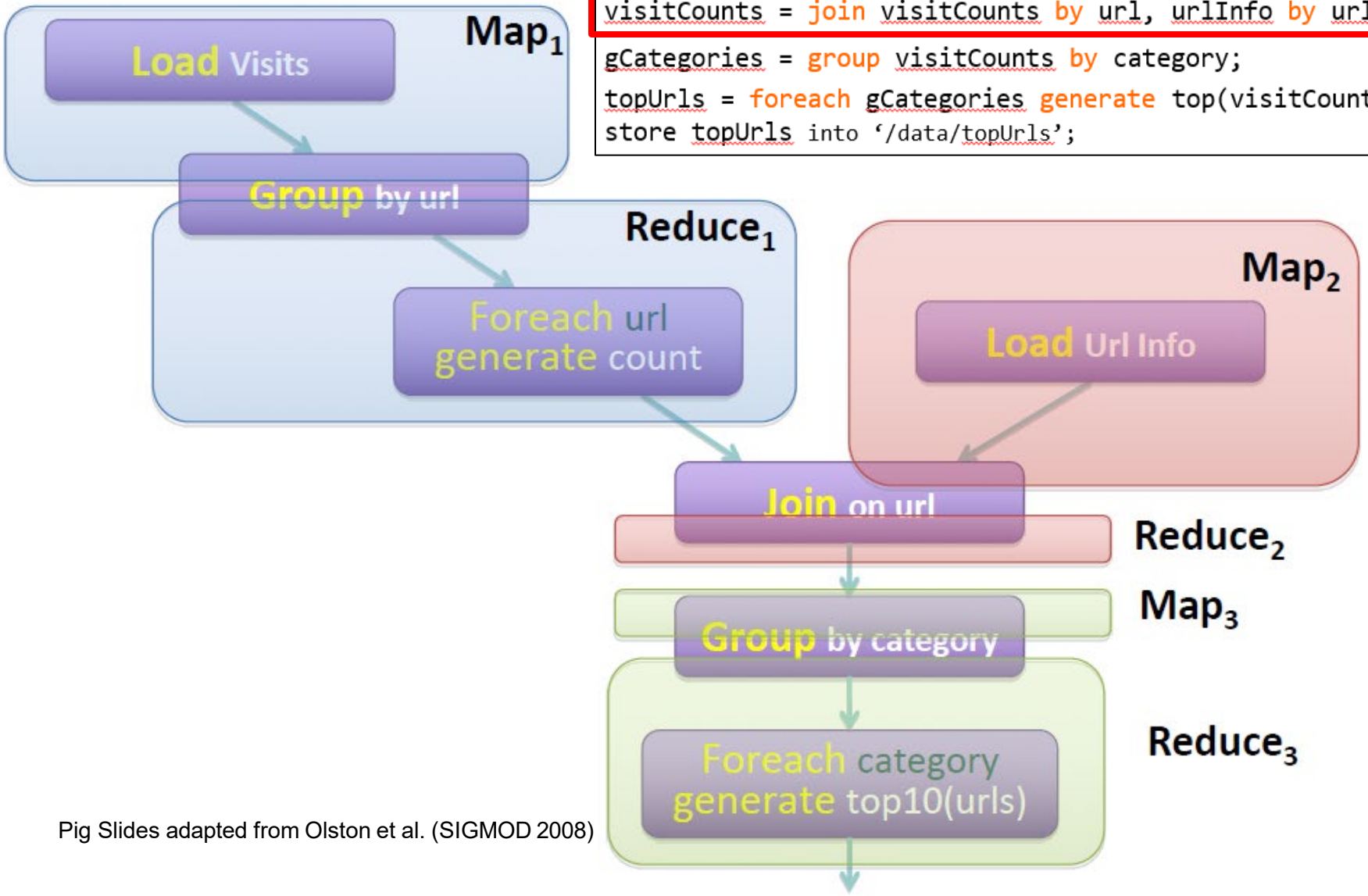
Pig Slides adapted from Olston et al. (SIGMOD 2008)

```
visits = load '/data/visits' as (user, url, time);
gVisits = group visits by url;
visitCounts = foreach gVisits generate url, count(visits);
urlInfo = load '/data/urlInfo' as (url, category, pRank);
visitCounts = join visitCounts by url, urlInfo by url;
gCategories = group visitCounts by category;
topUrls = foreach gCategories generate top(visitCounts,10);
store topUrls into '/data/topUrls';
```



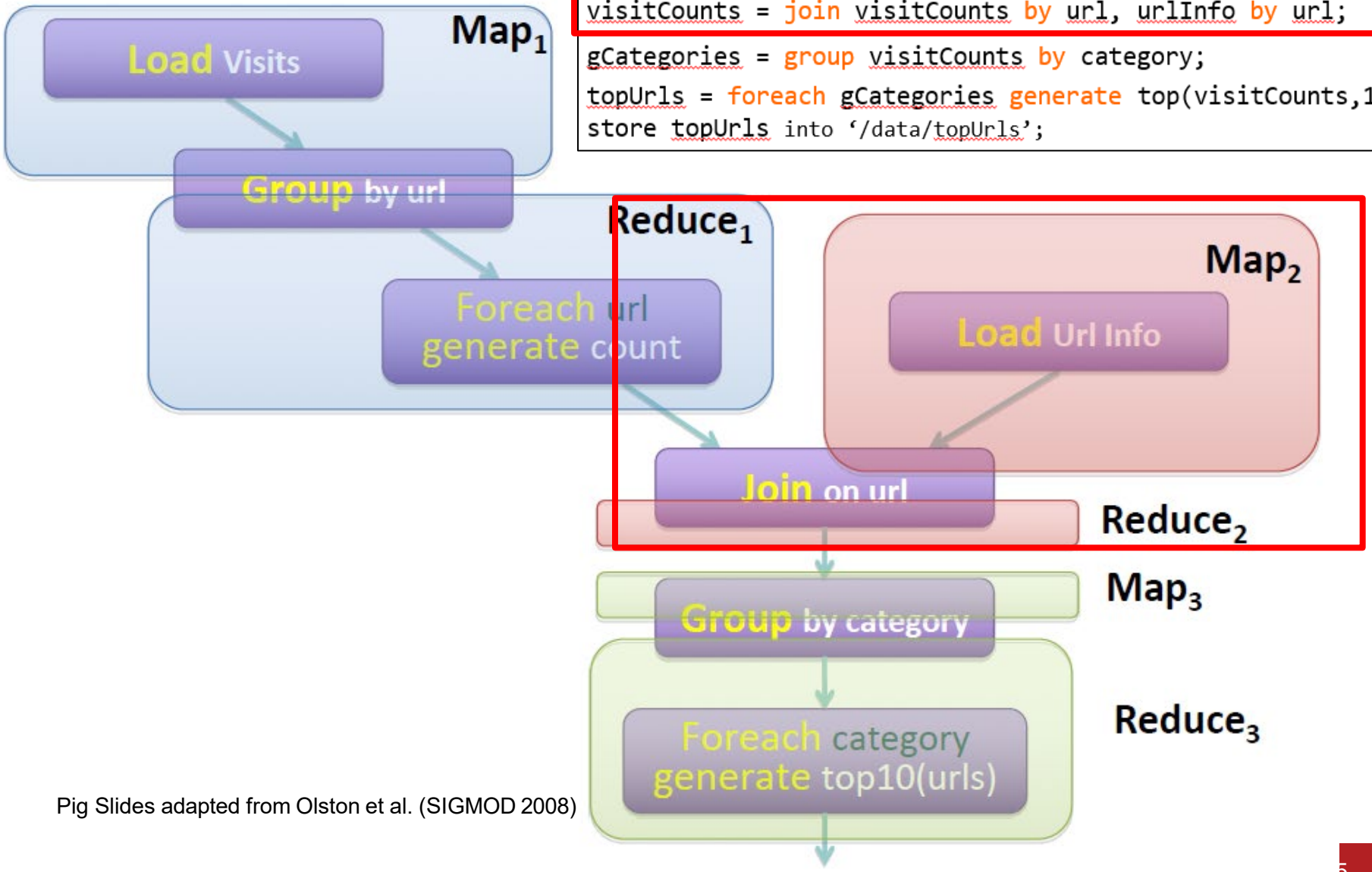
Pig Slides adapted from Olston et al. (SIGMOD 2008)

```
visits = load '/data/visits' as (user, url, time);
gVisits = group visits by url;
visitCounts = foreach gVisits generate url, count(visits);
urlInfo = load '/data/urlInfo' as (url, category, pRank);
visitCounts = join visitCounts by url, urlInfo by url;
gCategories = group visitCounts by category;
topUrls = foreach gCategories generate top(visitCounts,10);
store topUrls into '/data/topUrls';
```



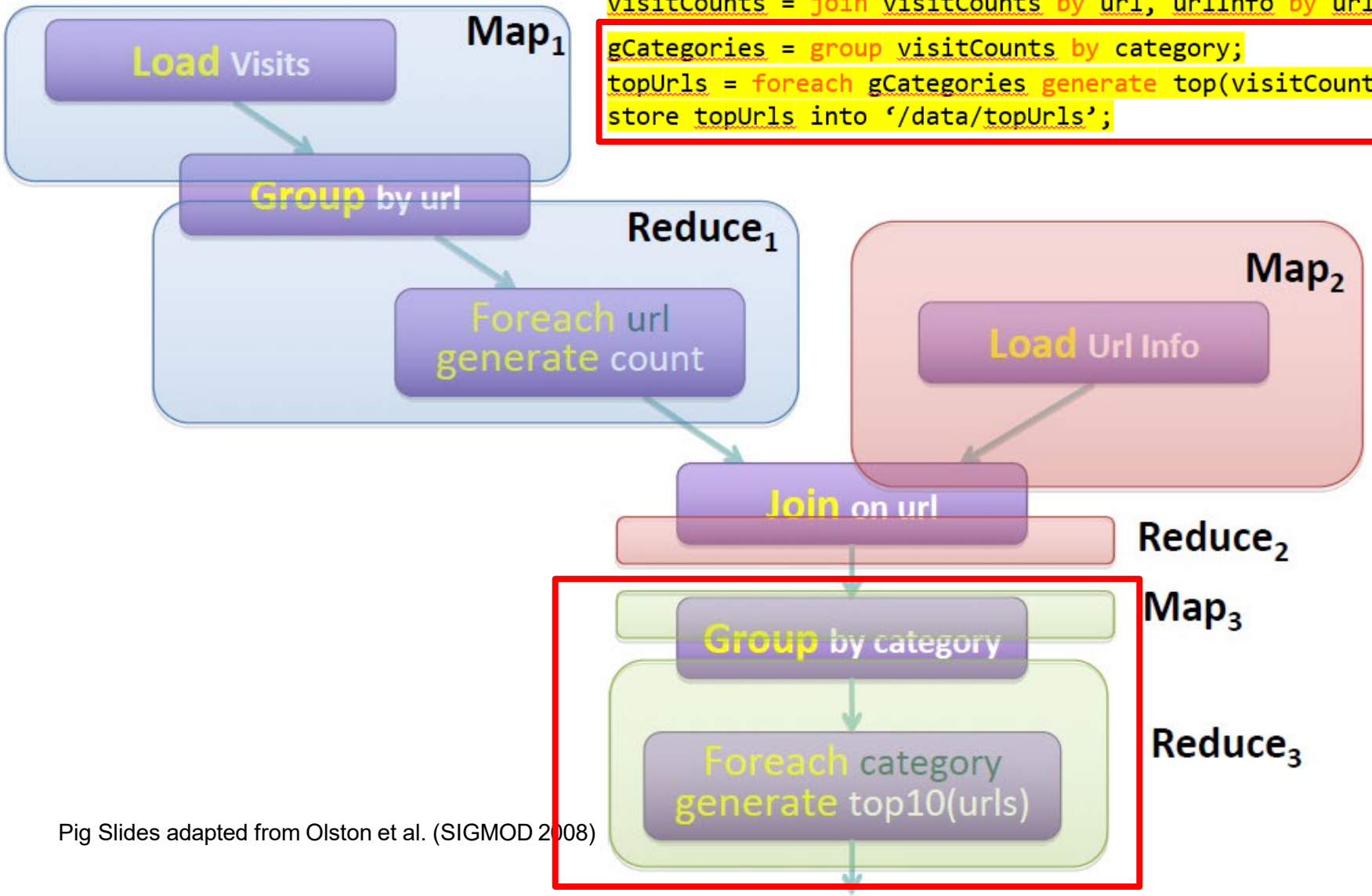
Pig Slides adapted from Olston et al. (SIGMOD 2008)

```
visits = load '/data/visits' as (user, url, time);
gVisits = group visits by url;
visitCounts = foreach gVisits generate url, count(visits);
urlInfo = load '/data/urlInfo' as (url, category, pRank);
visitCounts = join visitCounts by url, urlInfo by url;
gCategories = group visitCounts by category;
topUrls = foreach gCategories generate top(visitCounts,10);
store topUrls into '/data/topUrls';
```



Pig Slides adapted from Olston et al. (SIGMOD 2008)

```
visits = load '/data/visits' as (user, url, time);
gVisits = group visits by url;
visitCounts = foreach gVisits generate url, count(visits);
urlInfo = load '/data/urlInfo' as (url, category, pRank);
visitCounts = join visitCounts by url, urlInfo by url;
gCategories = group visitCounts by category;
topUrls = foreach gCategories generate top(visitCounts,10);
store topUrls into '/data/topUrls';
```



Pig Slides adapted from Olston et al. (SIGMOD 2008)

Q5 - Given the following pig script, which statements are correct?

1. visits = load 'hdfs:///user/ke/visits' using PigStorage(',') as (user, url, time);
2. gVisits = group visits by url;
3. visitCounts = foreach gVisits generate group as url, COUNT(visits) as count;
4. urlInfo = load 'hdfs:///user/ke/urlInfo' using PigStorage(',') as (url, category, pRank);
5. joinInfo = join visitCounts by url, urlInfo by url;
6. gCategories = group joinInfo by category;
7. topUrls = foreach gCategories generate TOP(2,1,joinInfo);
8. store topUrls into 'hdfs:///user/ke/topUrls';

Q5 - Given the following pig script, which statements are correct?

```
1. visits = load 'hdfs:///user/ke/visits' using PigStorage(',') as (user, url, time);
2. gVisits = group visits by url;
3. visitCounts = foreach gVisits generate group as url, COUNT(visits) as count;
4. urlInfo = load 'hdfs:///user/ke/urlInfo' using PigStorage(',') as (url, category, pRank);
5.
6.
7.
8.
```

https://pig.apache.org/docs/latest/func.html#topx

- TOP(2,1,joinInfo) - the two tuples in relation "joinInfo" with the highest value in the first column.
- Line 8 stores 10 tuples back to disk.
- Line 4 loads a data set where the default delimiter is applied.
- Line 2 groups the loaded tuples and outputs one tuple of visits for each url.
- Line 7 outputs 1 or 2 tuples of joinInfo for each category.

Q5 - Given the following pig script, which statements are correct?

```
1. visits = load 'hdfs:///user/ke/visits' using PigStorage(',') as (user, url, time);
2. gVisits = group visits by url;
3. visitCounts = foreach gVisits generate group as url, COUNT(visits) as count;
4. urlInfo = load 'hdfs:///user/ke/urlInfo' using PigStorage(',') as (url, category, pRank);
5. joinInfo = join visitCounts by url, urlInfo by url;
6. gCategories = group joinInfo by category;
7. topUrls = foreach gCategories generate TOP(2,1,joinInfo);
8. store topUrls into 'hdfs:///user/ke/topUrls';
```



TOP(2,1,joinInfo) - the two tuples in relation "joinInfo" with the highest value in the first column.

- ❑ Line 8 stores 10 tuples back to disk.
- ❑ Line 4 loads a data set where the default delimiter is applied.
- ❑ Line 2 groups the loaded tuples and outputs one tuple of visits for each url.
- ❑ Line 7 outputs 1 or 2 tuples of joinInfo for each category.

Q5 - Given the following pig script, which statements are correct?

```
1. visits = load 'hdfs:///user/ke/visits' using PigStorage(',') as (user, url, time);
2. gVisits = group visits by url;
3. visitCounts = foreach gVisits generate group as url, COUNT(visits) as count;
4. urlInfo = load 'hdfs:///user/ke/urlInfo' using PigStorage(',') as (url, category, pRank);
5. joinInfo = join visitCounts by url, urlInfo by url;
6. gCategories = group joinInfo by category;
7. topUrls = foreach gCategories generate TOP(2,1,joinInfo);
8. store topUrls into 'hdfs:///user/ke/topUrls';
```

- ✗ TOP(2,1,joinInfo) - the two tuples in relation "joinInfo" with the highest value in the first column.
- ✗ Line 8 stores 10 tuples back to disk.
- ❑ Line 4 loads a data set where the default delimiter is applied.
- ❑ Line 2 groups the loaded tuples and outputs one tuple of visits for each url.
- ❑ Line 7 outputs 1 or 2 tuples of joinInfo for each category.

Given the following pig script, which statements are correct?

```
1. visits = load 'hdfs:///user/ke/visits' using PigStorage(',') as (user, url, time);
2. gVisits = group visits by url;
3. visitCounts = foreach gVisits generate group as url, COUNT(visits) as count;
4. urlInfo = load 'hdfs:///user/ke/urlInfo' using PigStorage(',') as (url, category, pRank);
5. joinInfo = join visitCounts by url, urlInfo by url;
6. gCategories = group joinInfo by category;
7. topUrls = foreach gCategories generate TOP(2,1,joinInfo);
8. store topUrls into 'hdfs:///user/ke/topUrls';
```

https://pig.apache.org/docs/latest/func.html#pigstorage

- Line 4 loads a data set where the default delimiter is applied.
- Line 2 groups the loaded tuples and outputs one tuple of visits for each url.
- Line 7 outputs 1 or 2 tuples of joinInfo for each category.

Q5 - Given the following pig script, which statements are correct?

```
1. visits = load 'hdfs:///user/ke/visits' using PigStorage(',') as (user, url, time);
2. gVisits = group visits by url;
3. visitCounts = foreach gVisits generate group as url, COUNT(visits) as count;
4. urlInfo = load 'hdfs:///user/ke/urlInfo' using PigStorage(',') as (url, category, pRank);
5. joinInfo = join visitCounts by url, urlInfo by url;
6. gCategories = group joinInfo by category;
7. topUrls = foreach gCategories generate TOP(2,1,joinInfo);
8. store topUrls into 'hdfs:///user/ke/topUrls';
```

- ✗ TOP(2,1,joinInfo) - the two tuples in relation "joinInfo" with the highest value in the first column.
- ✗ Line 8 stores 10 tuples back to disk.
- ✗ Line 4 loads a data set where the default delimiter is applied.
- ❑ Line 2 groups the loaded tuples and outputs one tuple of visits for each url.
- ❑ Line 7 outputs 1 or 2 tuples of joinInfo for each category.

Q5 - Given the following pig script, which statements are correct?

```
1. visits = load 'hdfs:///user/ke/visits' using PigStorage(',') as (user, url, time);
2. gVisits = group visits by url;
3. visitCounts = foreach gVisits generate group as url, COUNT(visits) as count;
4. urlInfo = load 'hdfs:///user/ke/urlInfo' using PigStorage(',') as (url, category, pRank);
5. joinInfo = join visitCounts by url, urlInfo by url;
6. gCategories = group joinInfo by category;
7. topUrls = foreach gCategories generate TOP(2,1,joinInfo);
8. store topUrls into 'hdfs:///user/ke/topUrls';
```

- ✗ TOP(2,1,joinInfo) - the two tuples in relation "joinInfo" with the highest value in the first column.
- ✗ Line 8 stores 10 tuples back to disk.
- ✗ Line 4 loads a data set where the default delimiter is applied.
- ✗ Line 2 groups the loaded tuples and outputs one tuple of visits for each url.
- ❑ Line 7 outputs 1 or 2 tuples of joinInfo for each category.

Q5 - Given the following pig script, which statements are correct?

```
1. visits = load 'hdfs:///user/ke/visits' using PigStorage(',') as (user, url, time);
2. gVisits = group visits by url;
3. visitCounts = foreach gVisits generate group as url, COUNT(visits) as count;
4. urlInfo = load 'hdfs:///user/ke/urlInfo' using PigStorage(',') as (url, category, pRank);
5. joinInfo = join visitCounts by url, urlInfo by url;
6. gCategories = group joinInfo by category;
7. topUrls = foreach gCategories generate TOP(2,1,joinInfo);
8. store topUrls into 'hdfs:///user/ke/topUrls';
```

- ✗ TOP(2,1,joinInfo) - the two tuples in relation "joinInfo" with the highest value in the first column.
- ✗ Line 8 stores 10 tuples back to disk.
- ✗ Line 4 loads a data set where the default delimiter is applied.
- ✗ Line 2 groups the loaded tuples and outputs one tuple of visits for each url.
- ✓ Line 7 outputs 1 or 2 tuples of joinInfo for each category.

Questions?